



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

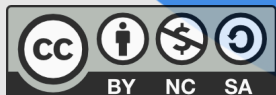


MaratonaCIn

Variações de Segment Tree

Lazy, Dual, Multi-op, Esparsa,
Persistente & Beats

rev. 1.0



Lua Guimarães Fernandes <lgf>

SegTree Comum

Range Query Point Update

O que é uma SegTree?



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

A Segment Tree (SegTree) é uma estrutura de dados que permite realizar consultas de intervalo e atualizações de elementos de forma eficiente.

- **Estrutura:** Árvore binária quase completa.
- **Fundamento:** Cada nó representa um intervalo $[L, R]$.

- Divisão: Se um nó representa $[L, R]$, seus filhos representam $[L, mid]$ e $[mid + 1, R]$, onde:

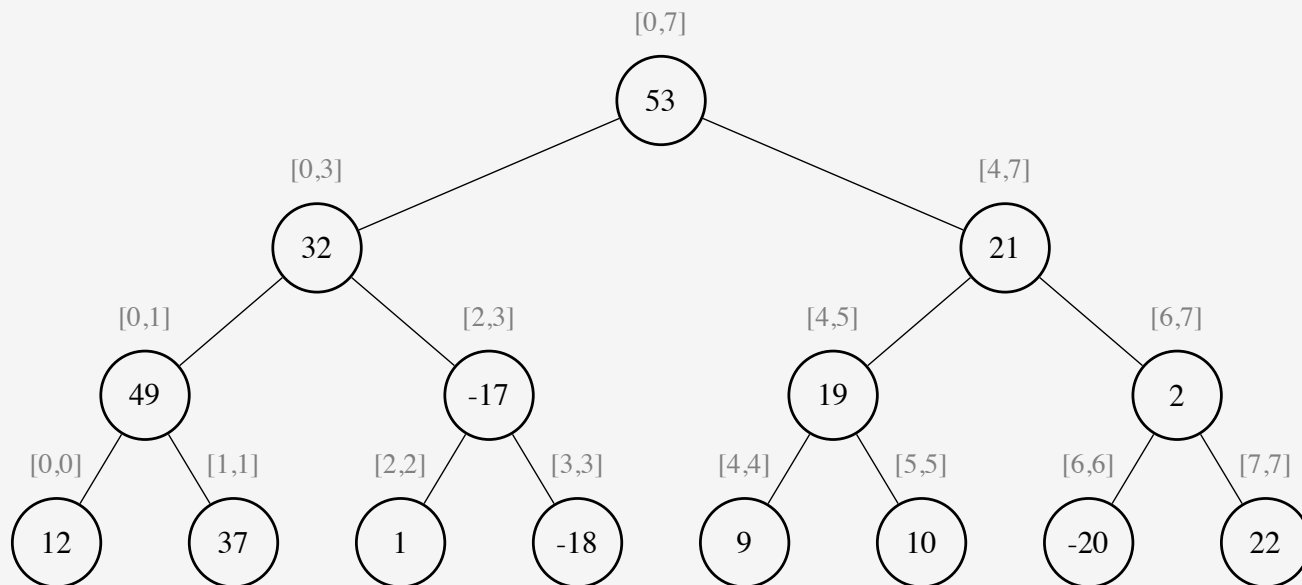
$$mid = L + \lfloor (R - L) / 2 \rfloor$$

- Complexidade:
 - Construção: $\mathcal{O}(n)$
 - Query/Update: $\mathcal{O}(\log n)$
 - Espaço: $\mathcal{O}(n)$

SegTree

Query: Soma no Intervalo $[1, 5]$

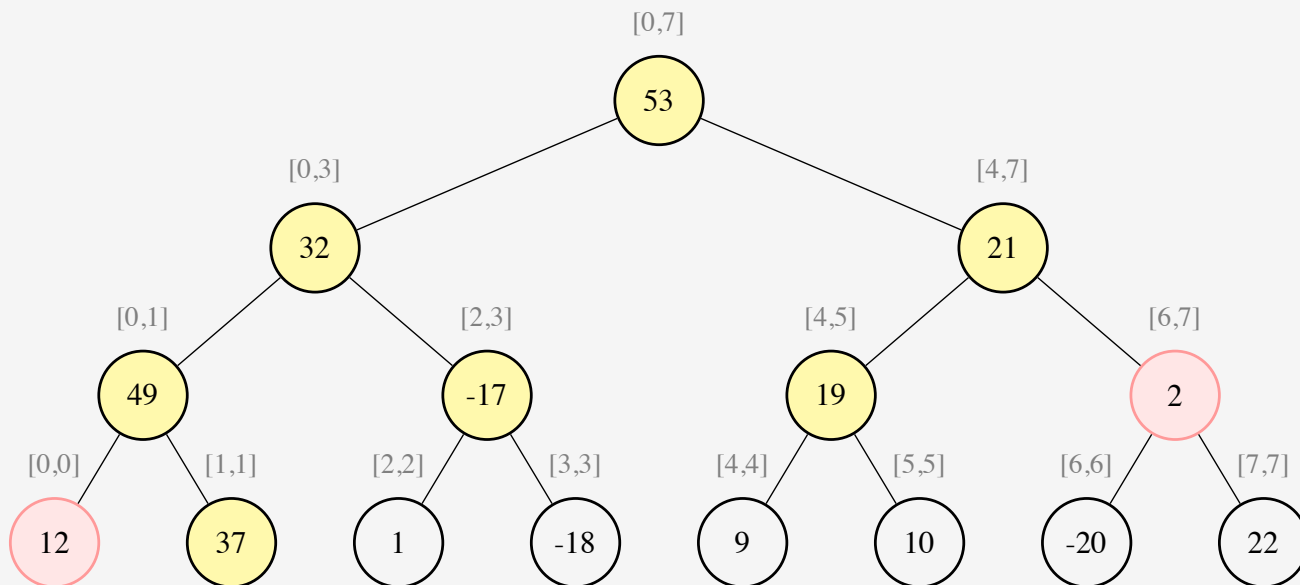
12	37	1	-18	9	10	-20	22
----	----	---	-----	---	----	-----	----



SegTree

Query: Soma no Intervalo $[1, 5]$

12	37	1	-18	9	10	-20	22
----	----	---	-----	---	----	-----	----



SegTree

Query: Soma no Intervalo $[1, 5]$

12

37

1

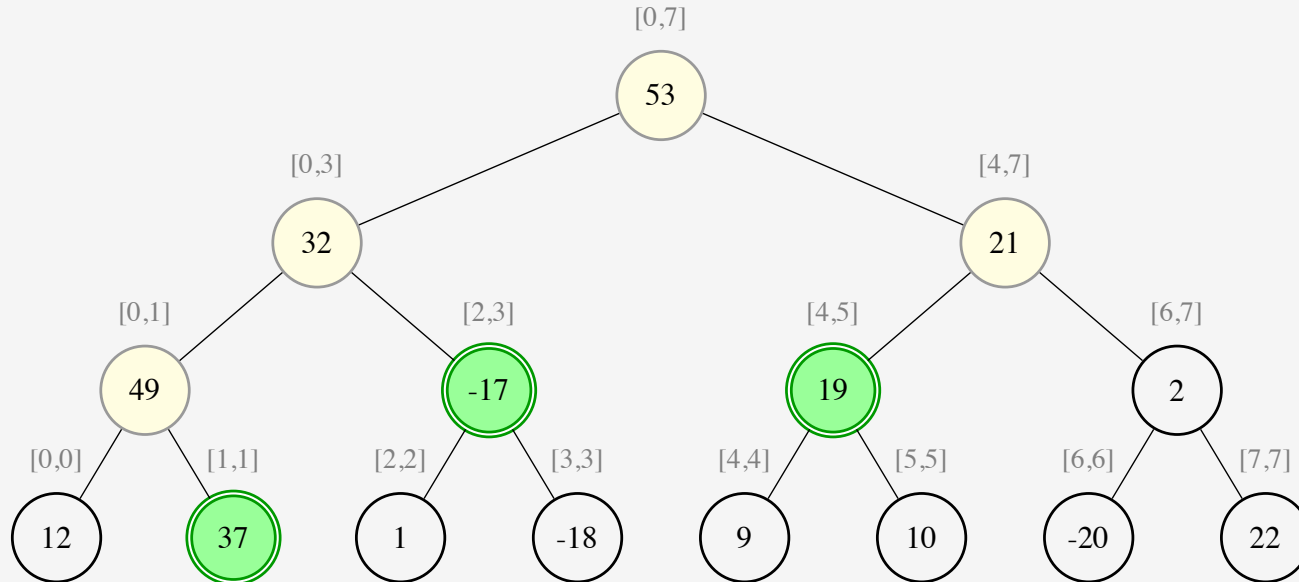
-18

9

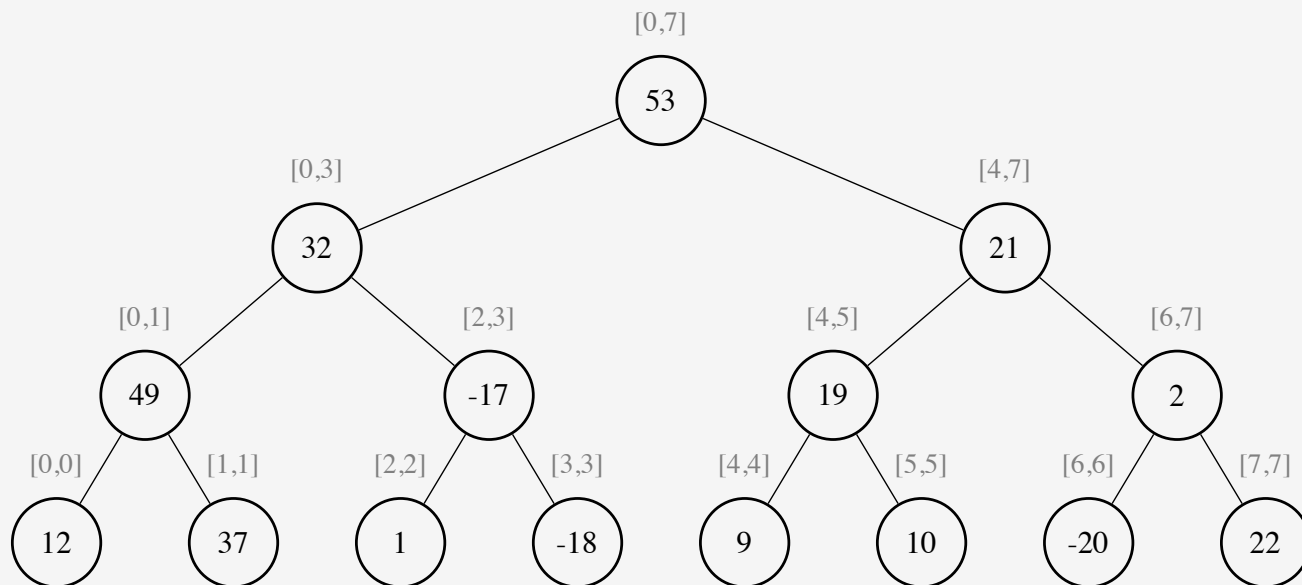
10

-20

22



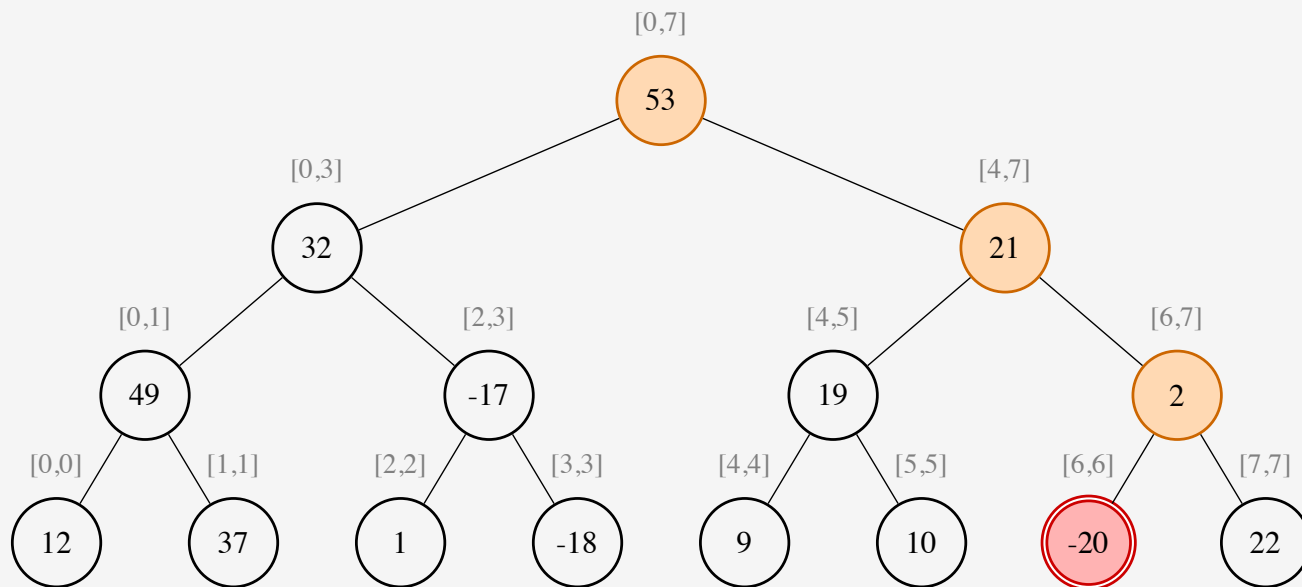
12	37	1	-18	9	10	-20	22
----	----	---	-----	---	----	-----	----



SegTree

Update: Alterar valor do índice 6

12	37	1	-18	9	10	-20	22
----	----	---	-----	---	----	-----	----



```
1  template<typename NODE>
2  struct SegTree {
3      int N;
4      vector<NODE> seg;
5
6      SegTree(int n) : N(n), seg(4 * n) {}
7
8      SegTree(const vector<int>& v) : N(v.size()), seg(4 * v.size()) {
9          build(1, 0, N - 1, v);
10     }
11
12     void build(int no, int l, int r, const vector<int>& v) {
13         if (l == r) {
14             seg[no] = NODE(v[l]);
15             return;
16         }
17         int m = (l + r) >> 1;
18         build(no << 1, l, m, v);
19         build((no << 1) | 1, m + 1, r, v);
```

```
1  template<typename NODE>
2  struct SegTree {
3      int N;
4      vector<NODE> seg;
5
6      SegTree(int n) : N(n), seg(4 * n) {}
7
8      SegTree(const vector<int>& v) : N(v.size()), seg(4 * v.size()) {
9          build(1, 0, N - 1, v);
10     }
11
12     void build(int no, int l, int r, const vector<int>& v) {
13         if (l == r) {
14             seg[no] = NODE(v[l]);
15             return;
16         }
17         int m = (l + r) >> 1;
18         build(no << 1, l, m, v);
```

```
1  template<typename NODE>
2  struct SegTree {
3      int N;
4      vector<NODE> seg;
5
6      SegTree(int n) : N(n), seg(4 * n) {}
7
8      SegTree(const vector<int>& v) : N(v.size()), seg(4 * v.size()) {
9          build(1, 0, N - 1, v);
10     }
11
12     void build(int no, int l, int r, const vector<int>& v) {
13         if (l == r) {
14             seg[no] = NODE(v[l]);
15             return;
16         }
17         int m = (l + r) >> 1;
18         build(no << 1, l, m, v);
```

```
8     SegTree(const vector<int>& v) : N(v.size()), seg(4 * v.size()) {
9         build(1, 0, N - 1, v);
10    }
11
12    void build(int no, int l, int r, const vector<int>& v) {
13        if (l == r) {
14            seg[no] = NODE(v[l]);
15            return;
16        }
17        int m = (l + r) >> 1;
18        build(no << 1, l, m, v);
19        build((no << 1) | 1, m + 1, r, v);
20        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
21    }
22
23    void update(int no, int l, int r, int idx, int val) {
24        if (l == r) {
25            seg[no].apply(val);
26            return;
27    }
```



```
8     SegTree(const vector<int>& v) : N(v.size()), seg(4 * v.size()) {
9         build(1, 0, N - 1, v);
10    }
11
12    void build(int no, int l, int r, const vector<int>& v) {
13        if (l == r) {
14            seg[no] = NODE(v[l]);
15            return;
16        }
17        int m = (l + r) >> 1;
18        build(no << 1, l, m, v);
19        build((no << 1) | 1, m + 1, r, v);
20        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
21    }
22
23    void update(int no, int l, int r, int idx, int val) {
24        if (l == r) {
25            seg[no].apply(val);
26            return;
27    }
```

```
19     build((no << 1) | 1, m + 1, r, v);
20     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
21 }
22
23 void update(int no, int l, int r, int idx, int val) {
24     if (l == r) {
25         seg[no].apply(val);
26         return;
27     }
28     int m = (l + r) >> 1;
29     if (idx <= m) update(no << 1, l, m, idx, val);
30     else update((no << 1) | 1, m + 1, r, idx, val);
31     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
32 }
33
34 NODE query(int no, int l, int r, int a, int b) {
35     if (b < l || r < a) return NODE();
36     if (a <= l && r <= b) return seg[no];
37     int m = (l + r) >> 1;
```

```
19     build((no << 1) | 1, m + 1, r, v);
20     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
21 }
22
23 void update(int no, int l, int r, int idx, int val) {
24     if (l == r) {
25         seg[no].apply(val);
26         return;
27     }
28     int m = (l + r) >> 1;
29     if (idx <= m) update(no << 1, l, m, idx, val);
30     else update((no << 1) | 1, m + 1, r, idx, val);
31     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
32 }
33
34 NODE query(int no, int l, int r, int a, int b) {
35     if (b < l || r < a) return NODE();
36     if (a <= l && r <= b) return seg[no];
37     int m = (l + r) >> 1;
```

```
27     }
28     int m = (l + r) >> 1;
29     if (idx ≤ m) update(no << 1, l, m, idx, val);
30     else update((no << 1) | 1, m + 1, r, idx, val);
31     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
32 }
33
34 NODE query(int no, int l, int r, int a, int b) {
35     if (b < l || r < a) return NODE();
36     if (a ≤ l && r ≤ b) return seg[no];
37     int m = (l + r) >> 1;
38     return NODE::merge(query(no << 1, l, m, a, b),
39                        query((no << 1) | 1, m + 1, r, a, b));
40 }
41
42 void update(int idx, int val) { update(1, 0, N - 1, idx, val); }
43 NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
44 };
```

```
27     }
28     int m = (l + r) >> 1;
29     if (idx ≤ m) update(no << 1, l, m, idx, val);
30     else update((no << 1) | 1, m + 1, r, idx, val);
31     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
32 }
33
34 NODE query(int no, int l, int r, int a, int b) {
35     if (b < l || r < a) return NODE();
36     if (a ≤ l && r ≤ b) return seg[no];
37     int m = (l + r) >> 1;
38     return NODE::merge(query(no << 1, l, m, a, b),
39                        query((no << 1) | 1, m + 1, r, a, b));
40 }
41
42 void update(int idx, int val) { update(1, 0, N - 1, idx, val); }
43 NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
44 };
```

```
27     }
28     int m = (l + r) >> 1;
29     if (idx ≤ m) update(no << 1, l, m, idx, val);
30     else update((no << 1) | 1, m + 1, r, idx, val);
31     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
32 }
33
34 NODE query(int no, int l, int r, int a, int b) {
35     if (b < l || r < a) return NODE();
36     if (a ≤ l && r ≤ b) return seg[no];
37     int m = (l + r) >> 1;
38     return NODE::merge(query(no << 1, l, m, a, b),
39                        query((no << 1) | 1, m + 1, r, a, b));
40 }
41
42 void update(int idx, int val) { update(1, 0, N - 1, idx, val); }
43 NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
44 };
```

```
1  template<typename NODE>
2  struct SegTree {
3      int N;
4      vector<NODE> seg;
5
6      SegTree(int n) : N(n), seg(4 * n) {}
7
8      SegTree(const vector<int>& v) : N(v.size()), seg(4 * v.size()) {
9          build(1, 0, N - 1, v);
10     }
11
12     void build(int no, int l, int r, const vector<int>& v) {
13         if (l == r) {
14             seg[no] = NODE(v[l]);
15             return;
16         }
17         int m = (l + r) >> 1;
18         build(no << 1, l, m, v);
19         build((no << 1) | 1, m + 1, r, v);
```

Seg Node

O Node é um template que guarda as informações de cada nó da segtree.

O Node tem 3 responsabilidades:

- Guardar seu próprio valor
- Saber se unir com outro nó
- Saber como aplicar um update

```
1  struct Node {
2      ll val = 0;
3      Node(ll v = 0) : val(v) {}
4
5      static inline Node merge(const Node& l, const Node& r) {
6          return Node(l.val + r.val);
7      }
8
9      inline void apply(int v) {
10         val += v;
11     }
12 };
```


Seg Node

O Node é um template que guarda as informações de cada nó da segtree.

O Node tem 3 responsabilidades:

- Guardar seu próprio valor
- Saber se unir com outro nó
- Saber como aplicar um update

```
1  struct Node {
2      ll val = 0;
3      Node(ll v = 0) : val(v) {}
4
5      static inline Node merge(const Node& l, const Node& r) {
6          return Node(l.val + r.val);
7      }
8
9      inline void apply(int v) {
10         val += v;
11     }
12 };
```

Seg Node

O Node é um template que guarda as informações de cada nó da segtree.

O Node tem 3 responsabilidades:

- Guardar seu próprio valor
- Saber se unir com outro nó
- Saber como aplicar um update

```
1  struct Node {
2      ll val = 0;
3      Node(ll v = 0) : val(v) {}
4
5      static inline Node merge(const Node& l, const Node& r) {
6          return Node(l.val + r.val);
7      }
8
9      inline void apply(int v) {
10         val += v;
11     }
12 };
```

Seg Node

O Node é um template que guarda as informações de cada nó da segtree.

O Node tem 3 responsabilidades:

- Guardar seu próprio valor
- Saber se unir com outro nó
- Saber como aplicar um update

```
1  struct Node {
2      ll val = 0;
3      Node(ll v = 0) : val(v) {}
4
5      static inline Node merge(const Node& l, const Node& r) {
6          return Node(l.val + r.val);
7      }
8
9      inline void apply(int v) {
10         val += v;
11     }
12 };
```

Seg Node

O Node é um template que guarda as informações de cada nó da segtree.

O Node tem 3 responsabilidades:

- Guardar seu próprio valor
- Saber se unir com outro nó
- Saber como aplicar um update

```
1  struct Node {
2      ll val = 0;
3      Node(ll v = 0) : val(v) {}
4
5      static inline Node merge(const Node& l, const Node& r) {
6          return Node(l.val + r.val);
7      }
8
9      inline void apply(int v) {
10         val += v;
11     }
12 };
```

Descida na Árvore (SegTree Walk)

É uma técnica para buscar um índice que satisfaz uma condição específica, navegando diretamente pela estrutura da árvore.

- O Problema: Encontrar o primeiro índice i tal que $A[i] \geq x$.
- Abordagem Clássica: Busca Binária na resposta + Range Query $\rightarrow \mathcal{O}(\log^2 n)$.
- Walk on Tree: Elimina a busca binária externa, descendo direto da raiz à folha $\rightarrow \mathcal{O}(\log n)$.

Como funciona (Lógica Gulosa):

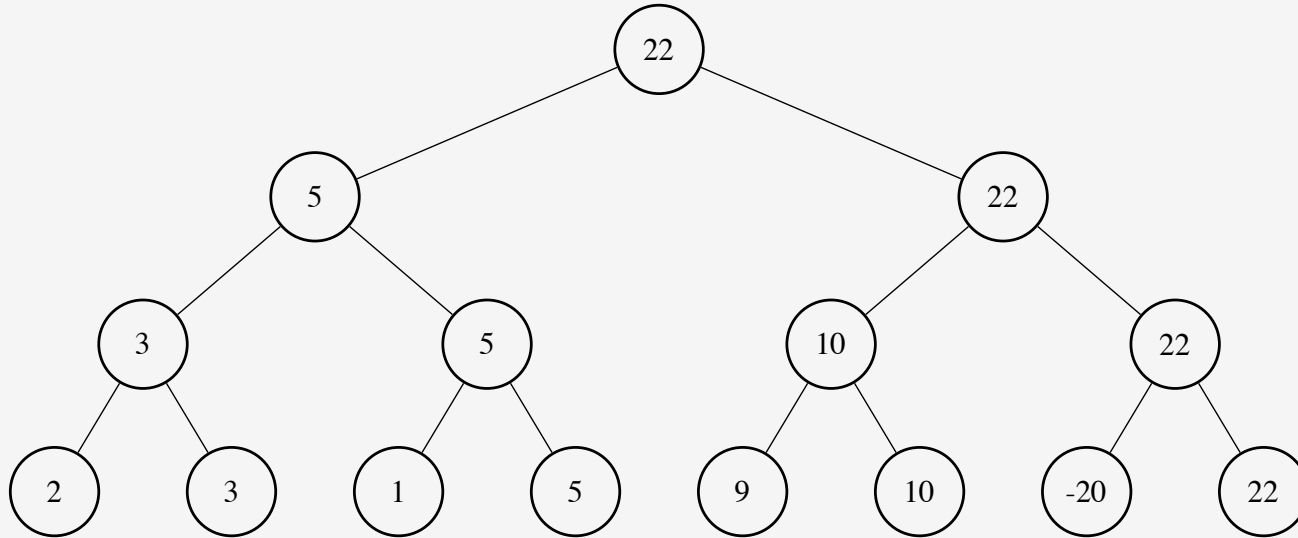
Em vez de quebrar o intervalo em vários nós, traçamos um caminho único:

- Começamos na Raiz.
- Verificamos o Filho da Esquerda:
- A resposta pode estar lá?
(ex: $\max(left) \geq x$)
 - Sim: Descemos para a Esquerda.
 - Não: Descemos para a Direita.
- Repetimos até chegar numa Folha.

SegTree Walk (Max)

Query: Primeiro índice com val ≥ 10

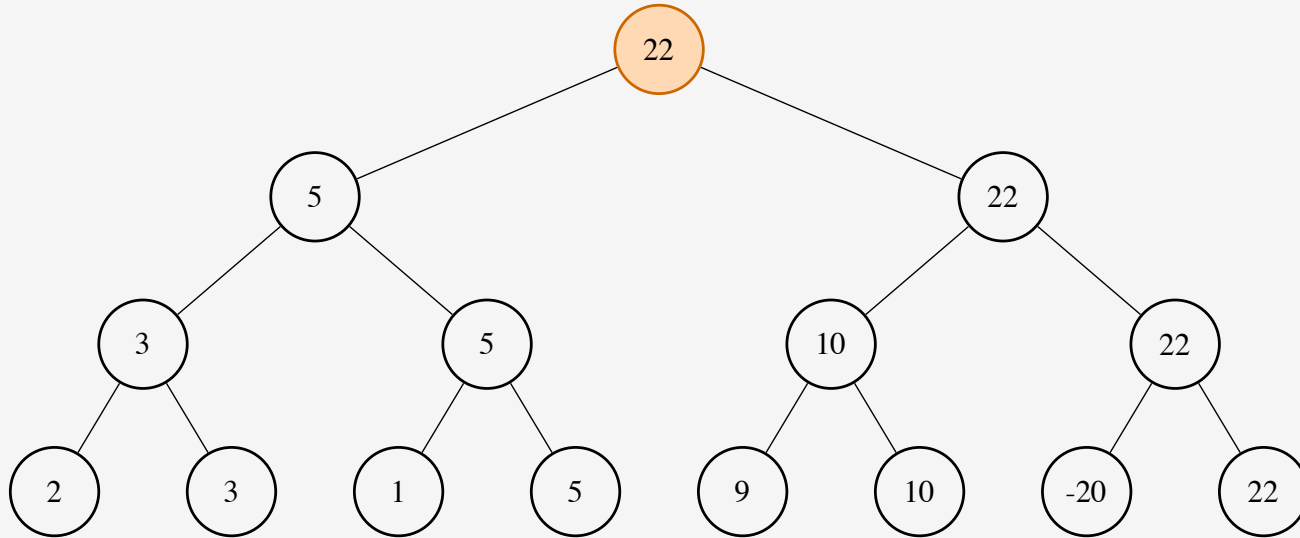
2	3	1	5	9	10	-20	22
---	---	---	---	---	----	-----	----



SegTree Walk (Max)

Query: Primeiro índice com val ≥ 10

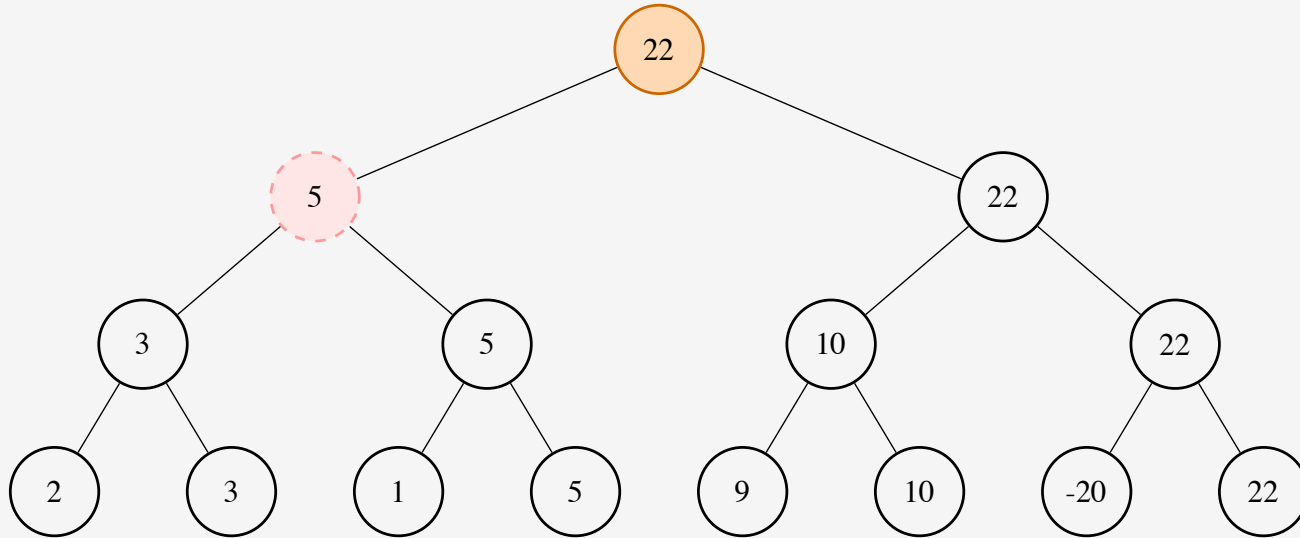
2	3	1	5	9	10	-20	22
---	---	---	---	---	----	-----	----



SegTree Walk (Max)

Query: Primeiro índice com val ≥ 10

2	3	1	5	9	10	-20	22
---	---	---	---	---	----	-----	----



SegTree Walk (Max)

Query: Primeiro índice com val ≥ 10



Centro de
Informática
UFPE

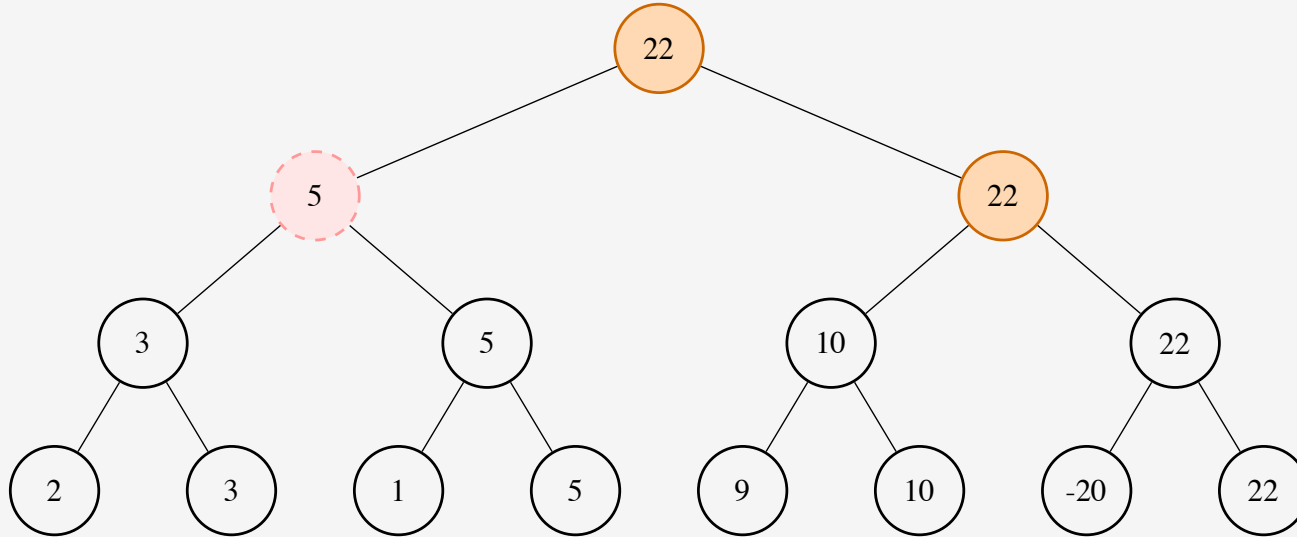


UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

2	3	1	5	9	10	-20	22
---	---	---	---	---	----	-----	----



SegTree Walk (Max)

Query: Primeiro índice com val ≥ 10



Centro de
Informática
UFPE

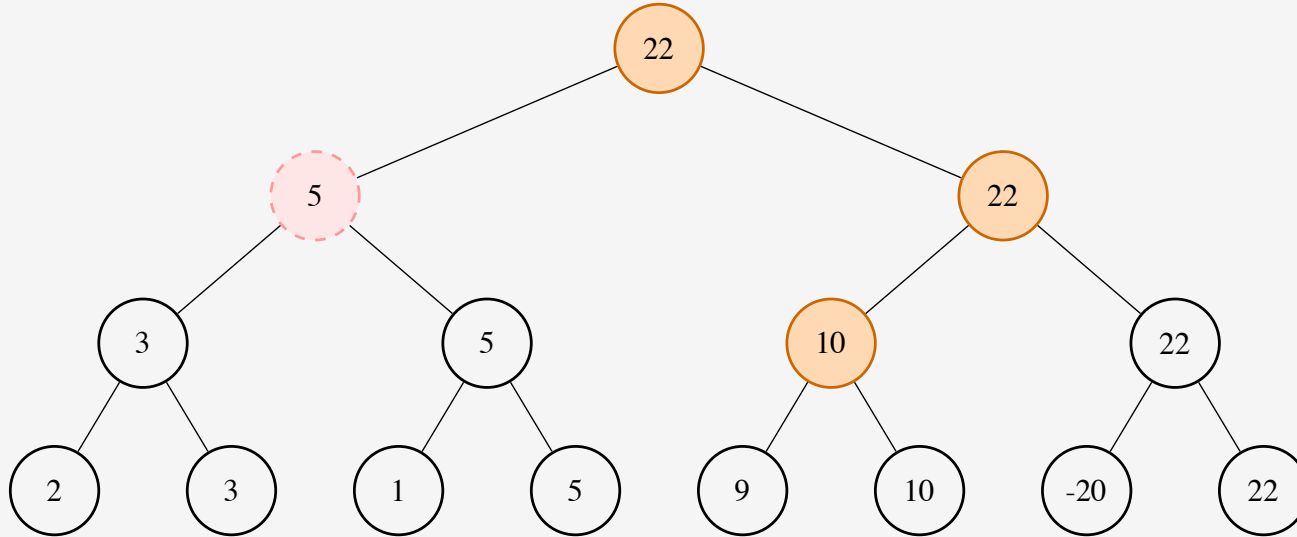


UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

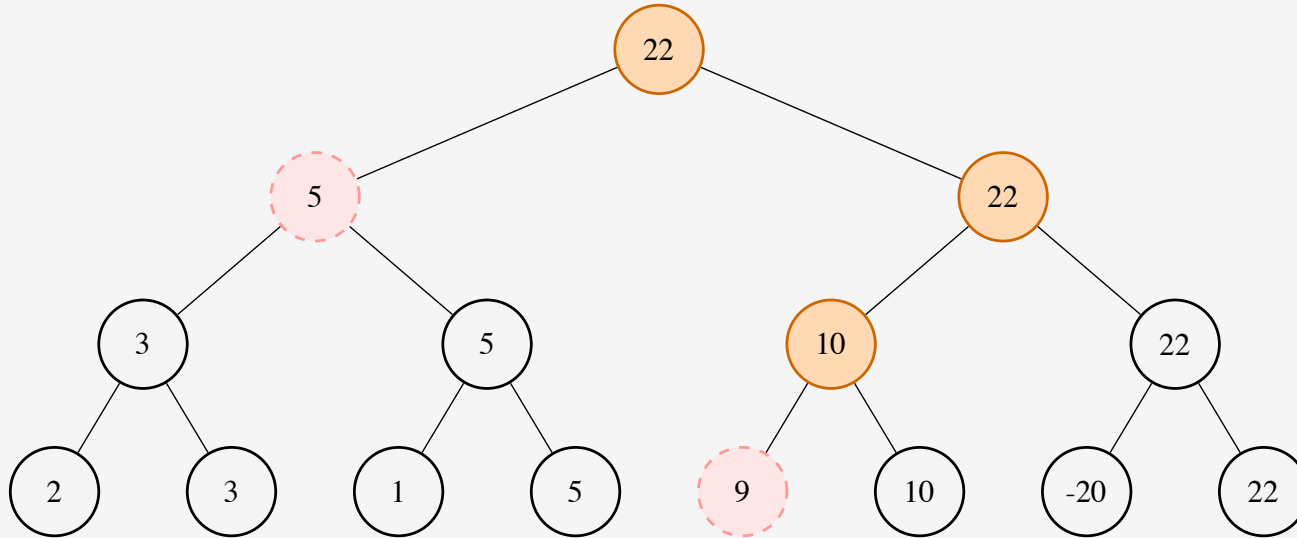
2	3	1	5	9	10	-20	22
---	---	---	---	---	----	-----	----



SegTree Walk (Max)

Query: Primeiro índice com val ≥ 10

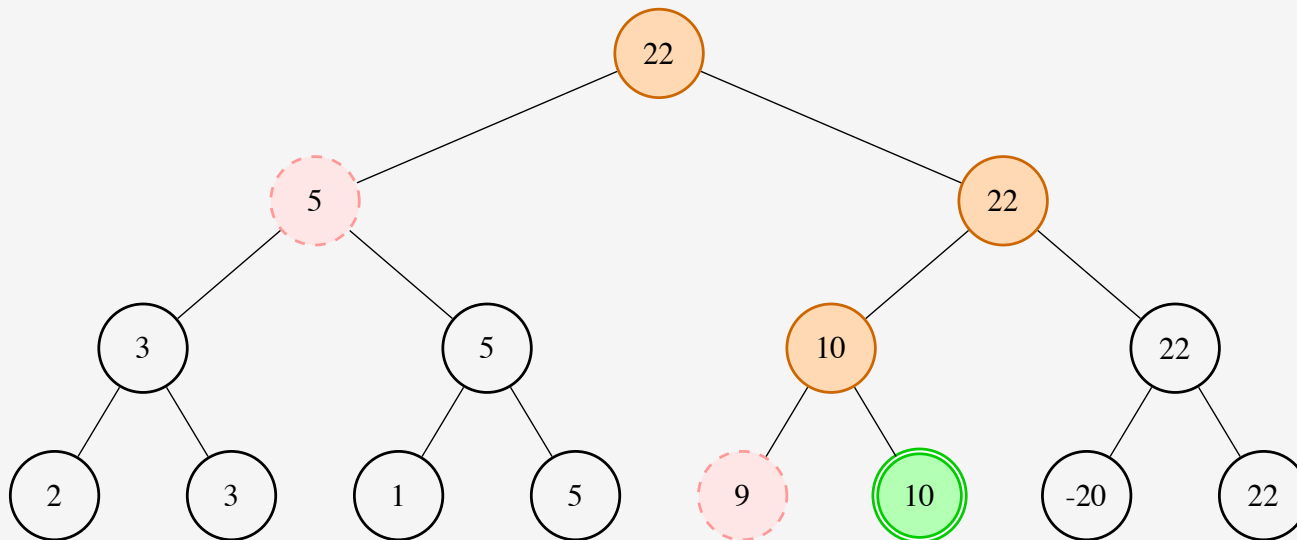
2	3	1	5	9	10	-20	22
---	---	---	---	---	----	-----	----



SegTree Walk (Max)

Query: Primeiro índice com val ≥ 10

2	3	1	5	9	10	-20	22
---	---	---	---	---	----	-----	----



```
1  int walk(SegTree<Node>& seg, int target, int no = 1, int l = 0, int r = -1) {
2      if (r == -1) r = seg.N - 1;
3      if (seg.seg[no].val < target) return -1;
4      if (l == r) return l;
5
6      int m = (l + r) >> 1;
7      int e = no << 1, d = e | 1;
8
9      if (seg.seg[e].val >= target)
10         return walk(seg, target, e, l, m);
11     else
12         return walk(seg, target, d, m + 1, r);
13 }
```

```
1  int walk(SegTree<Node>& seg, int target, int no = 1, int l = 0, int r = -1) {
2      if (r == -1) r = seg.N - 1;
3      if (seg.seg[no].val < target) return -1;
4      if (l == r) return l;
5
6      int m = (l + r) >> 1;
7      int e = no << 1, d = e | 1;
8
9      if (seg.seg[e].val >= target)
10         return walk(seg, target, e, l, m);
11     else
12         return walk(seg, target, d, m + 1, r);
13 }
```

```
1  int walk(SegTree<Node>& seg, int target, int no = 1, int l = 0, int r = -1) {
2      if (r == -1) r = seg.N - 1;
3      if (seg.seg[no].val < target) return -1;
4      if (l == r) return l;
5
6      int m = (l + r) >> 1;
7      int e = no << 1, d = e | 1;
8
9      if (seg.seg[e].val ≥ target)
10         return walk(seg, target, e, l, m);
11     else
12         return walk(seg, target, d, m + 1, r);
13 }
```

```
1  int walk(SegTree<Node>& seg, int target, int no = 1, int l = 0, int r = -1) {
2      if (r == -1) r = seg.N - 1;
3      if (seg.seg[no].val < target) return -1;
4      if (l == r) return l;
5
6      int m = (l + r) >> 1;
7      int e = no << 1, d = e | 1;
8
9      if (seg.seg[e].val >= target)
10         return walk(seg, target, e, l, m);
11     else
12         return walk(seg, target, d, m + 1, r);
13 }
```



```
1  int walk(SegTree<Node>& seg, int target, int no = 1, int l = 0, int r = -1) {
2      if (r == -1) r = seg.N - 1;
3      if (seg.seg[no].val < target) return -1;
4      if (l == r) return l;
5
6      int m = (l + r) >> 1;
7      int e = no << 1, d = e | 1;
8
9      if (seg.seg[e].val >= target)
10         return walk(seg, target, e, l, m);
11     else
12         return walk(seg, target, d, m + 1, r);
13 }
```

```
1  int walk(SegTree<Node>& seg, int target, int no = 1, int l = 0, int r = -1) {
2      if (r == -1) r = seg.N - 1;
3      if (seg.seg[no].val < target) return -1;
4      if (l == r) return l;
5
6      int m = (l + r) >> 1;
7      int e = no << 1, d = e | 1;
8
9      if (seg.seg[e].val >= target)
10         return walk(seg, target, e, l, m);
11     else
12         return walk(seg, target, d, m + 1, r);
13 }
```

Dúvidas?

SegTree Lazy

Range Query Range Update

Às vezes, point update não é suficiente.
Queremos poder adicionar um valor à
todo mundo no intervalo ou setar um
valor pra todo mundo no intervalo.

Para isso, podemos usar uma técnica
chamada de Lazy Propagation
(Propagação Preguiçosa).

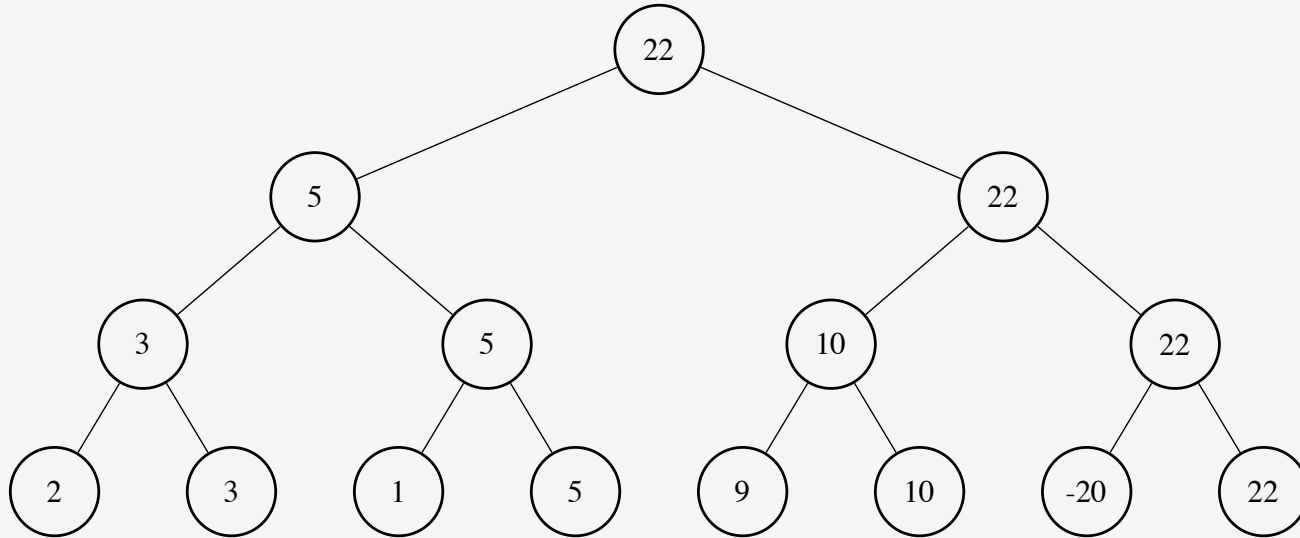
A ideia principal é que não precisamos
atualizar todo mundo de uma vez só.
Podemos ter updates pendentes em
cada nó e, somente quando
descermos em uma consulta,
propagamos a atualização.

Isso garante que cada update ainda
permaneça em $\mathcal{O}(\log n)$

SegTree Walk (Max)

Query: Primeiro índice com val ≥ 10

2	3	1	5	9	10	-20	22
---	---	---	---	---	----	-----	----



```
1  template<typename NODE>
2  struct SegTree {
3      int N;
4      vector<NODE> seg;
5
6      SegTree(int n) : N(n), seg(4 * n) {}
7
8      SegTree(const vector<int>& v) : N(v.size()), seg(4 * v.size()) {
9          build(1, 0, N - 1, v);
10     }
11
12     void build(int no, int l, int r, const vector<int>& v) {
13         if (l == r) {
14             seg[no] = NODE(v[l]);
15             return;
16         }
17         int m = (l + r) >> 1;
18         build(no << 1, l, m, v);
19         build((no << 1) | 1, m + 1, r, v);
20         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
21     }
```

```
1  template<typename NODE, typename TAG>
2  struct LazySegmentTree {
3      int N;
4      vector<NODE> seg;
5      vector<TAG> lazy;
6
7      LazySegmentTree(int n) : N(n), seg(4 * n), lazy(4 * n) {}
8
9      LazySegmentTree(const vector<int>& v)
10         : N(v.size()), seg(4 * v.size()), lazy(4 * v.size()) {
11         build(1, 0, N - 1, v);
12     }
13
14     void build(int no, int l, int r, const vector<int>& v) {
15         if (l == r) {
16             seg[no] = NODE(v[l]);
17             return;
18         }
19         int m = (l + r) >> 1;
20         build(no << 1, l, m, v);
21         build((no << 1) | 1, m + 1, r, v);
22         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
23     }
```



```
1 void update(int no, int l, int r, int idx, int val) { // SegTree
2     if (l == r) {
3         seg[no].apply(val);
4         return;
5     }
6     int m = (l + r) >> 1;
7     if (idx <= m) update(no << 1, l, m, idx, val);
8     else update((no << 1) | 1, m + 1, r, idx, val);
9     seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
10 }
```

```
1 void update(int no, int l, int r, int a, int b, const TAG& v) { // SegLazy
2     if (b < l || r < a) return;
3     if (a ≤ l && r ≤ b) {
4         seg[no].apply(v, l, r);
5         lazy[no].compose(v);
6         return;
7     }
8     push(no, l, r);
9     int m = (l + r) >> 1;
10    update(no << 1, l, m, a, b, v);
11    update((no << 1) | 1, m + 1, r, a, b, v);
12    seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
13 }
```

```
1  NODE query(int no, int l, int r, int a, int b) { // SegTree
2      if (b < l || r < a) return NODE();
3      if (a ≤ l && r ≤ b) return seg[no];
4      int m = (l + r) >> 1;
5      return NODE::merge(query(no << 1, l, m, a, b),
6                          query((no << 1) | 1, m + 1, r, a, b));
7  }
8
9  void update(int idx, int val) { update(1, 0, N - 1, idx, val); }
10 NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
```

```
1  NODE query(int no, int l, int r, int a, int b) { // SegLazy
2      if (b < l || r < a) return NODE();
3      if (a ≤ l && r ≤ b) return seg[no];
4      push(no, l, r);
5      int m = (l + r) >> 1;
6      return NODE::merge(query(no << 1, l, m, a, b),
7                          query((no << 1) | 1, m + 1, r, a, b));
8  }
9
10 void update(int l, int r, const TAG& v) { update(1, 0, N - 1, l, r, v); }
11 NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
```

```
1 void push(int no, int l, int r) { // SegLazy
2     int m = (l + r) >> 1;
3     int e = no << 1, d = e | 1;
4
5     seg[e].apply(lazy[no], l, m);
6     lazy[e].compose(lazy[no]);
7
8     seg[d].apply(lazy[no], m + 1, r);
9     lazy[d].compose(lazy[no]);
10
11     lazy[no] = TAG();
12 }
```

```
1 void push(int no, int l, int r) { // SegLazy
2     int m = (l + r) >> 1;
3     int e = no << 1, d = e | 1;
4
5     seg[e].apply(lazy[no], l, m);
6     lazy[e].compose(lazy[no]);
7
8     seg[d].apply(lazy[no], m + 1, r);
9     lazy[d].compose(lazy[no]);
10
11     lazy[no] = TAG();
12 }
```

```
1 void push(int no, int l, int r) { // SegLazy
2     int m = (l + r) >> 1;
3     int e = no << 1, d = e | 1;
4
5     seg[e].apply(lazy[no], l, m);
6     lazy[e].compose(lazy[no]);
7
8     seg[d].apply(lazy[no], m + 1, r);
9     lazy[d].compose(lazy[no]);
10
11     lazy[no] = TAG();
12 }
```

```
1 void push(int no, int l, int r) { // SegLazy
2     int m = (l + r) >> 1;
3     int e = no << 1, d = e | 1;
4
5     seg[e].apply(lazy[no], l, m);
6     lazy[e].compose(lazy[no]);
7
8     seg[d].apply(lazy[no], m + 1, r);
9     lazy[d].compose(lazy[no]);
10
11     lazy[no] = TAG();
12 }
```



```
1 void push(int no, int l, int r) { // SegLazy
2     int m = (l + r) >> 1;
3     int e = no << 1, d = e | 1;
4
5     seg[e].apply(lazy[no], l, m);
6     lazy[e].compose(lazy[no]);
7
8     seg[d].apply(lazy[no], m + 1, r);
9     lazy[d].compose(lazy[no]);
10
11     lazy[no] = TAG();
12 }
```

Seg Tag

A Tag é um template que guarda as informações de update para cada nó da segtree.

O Tag tem 2 responsabilidades:

- Guardar o update pendente
- Saber se unir com outra tag

O node da SegLazy precisa saber aplicar a tag em range.

```
1 struct Node { // SegTree
2     ll val = 0;
3     Node(ll v = 0) : val(v) {}
4
5     static inline Node merge(const Node& l, const Node& r) {
6         return Node(l.val + r.val);
7     }
8
9     inline void apply(int v) {
10         val += v;
11     }
12 };
```

Seg Tag

A Tag é um template que guarda as informações de update para cada nó da segtree.

O Tag tem 2 responsabilidades:

- Guardar o update pendente
- Saber se unir com outra tag

O node da SegLazy precisa saber aplicar a tag em range.

```
1 struct Node { // SegLazy
2     ll val = 0;
3     Node(ll v = 0) : val(v) {}
4
5     static inline Node merge(const Node& l, const Node& r) {
6         return Node(l.val + r.val);
7     }
8
9     inline void apply(const Tag& t, int l, int r) {
10         val = val + (r - l + 1) * t.add;
11     }
12 };
13
14 struct Tag {
15     ll add = 0;
16
17     inline void compose(const Tag& t) {
18         add += t.add;
19     }
20 };
```

Seg Tag

A Tag é um template que guarda as informações de update para cada nó da segtree.

O Tag tem 2 responsabilidades:

- Guardar o update pendente
- Saber se unir com outra tag

O node da SegLazy precisa saber aplicar a tag em range.

```
1 struct Node { // SegLazy
2     ll val = 0;
3     Node(ll v = 0) : val(v) {}
4
5     static inline Node merge(const Node& l, const Node& r) {
6         return Node(l.val + r.val);
7     }
8
9     inline void apply(const Tag& t, int l, int r) {
10         val = val + (r - l + 1) * t.add;
11     }
12 };
13
14 struct Tag {
15     ll add = 0;
16
17     inline void compose(const Tag& t) {
18         add += t.add;
19     }
20 };
```

Seg Tag



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

A Tag é um template que guarda as informações de update para cada nó da segtree.

O Tag tem 2 responsabilidades:

- Guardar o update pendente
- Saber se unir com outra tag

O node da SegLazy precisa saber aplicar a tag em range.

```
1 struct Node { // SegLazy
2     ll val = 0;
3     Node(ll v = 0) : val(v) {}
4
5     static inline Node merge(const Node& l, const Node& r) {
6         return Node(l.val + r.val);
7     }
8
9     inline void apply(const Tag& t, int l, int r) {
10         val = val + (r - l + 1) * t.add;
11     }
12 };
13
14 struct Tag {
15     ll add = 0;
16
17     inline void compose(const Tag& t) {
18         add += t.add;
19     }
20 };
```

Seg Tag

A Tag é um template que guarda as informações de update para cada nó da segtree.

O Tag tem 2 responsabilidades:

- Guardar o update pendente
- Saber se unir com outra tag

O node da SegLazy precisa saber aplicar a tag em range.

```
1 struct Node { // SegLazy
2     ll val = 0;
3     Node(ll v = 0) : val(v) {}
4
5     static inline Node merge(const Node& l, const Node& r) {
6         return Node(l.val + r.val);
7     }
8
9     inline void apply(const Tag& t, int l, int r) {
10         val = val + (r - l + 1) * t.add;
11     }
12 };
13
14 struct Tag {
15     ll add = 0;
16
17     inline void compose(const Tag& t) {
18         add += t.add;
19     }
20 };
```

Dúvidas?

SegTree Lazy Dual

Point Query Range Update

Seg Lazy Dual



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Às vezes, precisamos de range update, mas as queries serão só de pontos. Podemos usar uma Seg Lazy normal para isso, no entanto, a Lazy Dual é mais eficiente!

A Lazy Dual é uma variação da SegTree Lazy onde, ao invés de armazenarmos os valores parciais nos nós, armazenamos apenas os updates pendentes e os valores das folhas.

A ideia principal é que não precisamos guardar todos os nós da seg, apenas as folhas. Assim não precisamos atualizar os nós no update, só dar push no lazy tag.

A complexidade se mantém a mesma, no entanto, a constante acaba melhorando pois precisamos de menos operações intermediárias por update.

```
1  template<typename NODE, typename TAG>
2  struct LazySegmentTree {
3      int N;
4      vector<NODE> seg;
5      vector<TAG> lazy;
6
7      LazySegmentTree(int n) : N(n), seg(4 * n), lazy(4 * n) {}
8
9      LazySegmentTree(const vector<int>& v)
10         : N(v.size()), seg(4 * v.size()), lazy(4 * v.size()) {
11         build(1, 0, N - 1, v);
12     }
13
14     void build(int no, int l, int r, const vector<int>& v) {
15         if (l == r) {
16             seg[no] = NODE(v[l]);
17             return;
18         }
19         int m = (l + r) >> 1;
20         build(no << 1, l, m, v);
21         build((no << 1) | 1, m + 1, r, v);
22         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
23     }
```

```
1  template<typename T, typename TAG>
2  struct DualSegTree {
3      int N;
4      vector<TAG> lazy;
5      vector<T> leaves;
6
7      DualSegTree(int n) : N(n), lazy(4 * n), leaves(n) {}
8      DualSegTree(const vector<T>& v)
9          : N(v.size()), lazy(4 * v.size()), leaves(v) {}
```

```
1 void update(int no, int l, int r, int a, int b, const TAG& v) { // SegLazy
2     if (b < l || r < a) return;
3     if (a ≤ l && r ≤ b) {
4         seg[no].apply(v, l, r);
5         lazy[no].compose(v);
6         return;
7     }
8     push(no, l, r);
9     int m = (l + r) >> 1;
10    update(no << 1, l, m, a, b, v);
11    update((no << 1) | 1, m + 1, r, a, b, v);
12    seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
13 }
```

```
1 void update(int no, int l, int r, int a, int b, const TAG& v) { // SegDual
2     if (b < l || r < a) return;
3     if (a ≤ l && r ≤ b) {
4         lazy[no].compose(v);
5         return;
6     }
7     push(no, l, r);
8     int m = (l + r) >> 1;
9     update(no << 1, l, m, a, b, v);
10    update((no << 1) | 1, m + 1, r, a, b, v);
11 }
```

```
1  NODE query(int no, int l, int r, int a, int b) { // SegLazy
2      if (b < l || r < a) return NODE();
3      if (a ≤ l && r ≤ b) return seg[no];
4      push(no, l, r);
5      int m = (l + r) >> 1;
6      return NODE::merge(query(no << 1, l, m, a, b),
7                          query((no << 1) | 1, m + 1, r, a, b));
8  }
9
10 void update(int l, int r, const TAG& v) { update(1, 0, N - 1, l, r, v); }
11 NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
```

```
1  T get(int no, int l, int r, int idx) { // SegDual
2      if (l == r) {
3          T res = leaves[idx];
4          lazy[no].apply(res);
5          return res;
6      }
7      push(no, l, r);
8      int m = (l + r) >> 1;
9      if (idx <= m) return get(no << 1, l, m, idx);
10     else return get((no << 1) | 1, m + 1, r, idx);
11 }
12
13 void update(int l, int r, const TAG& t) { update(1, 0, N - 1, l, r, t); }
14 T get(int idx) { return get(1, 0, N - 1, idx); }
```

```
1 void push(int no, int l, int r) { // SegLazy
2     int m = (l + r) >> 1;
3     int e = no << 1, d = e | 1;
4
5     seg[e].apply(lazy[no], l, m);
6     lazy[e].compose(lazy[no]);
7
8     seg[d].apply(lazy[no], m + 1, r);
9     lazy[d].compose(lazy[no]);
10
11     lazy[no] = TAG();
12 }
```



```
1 void push(int no, int l, int r) { // SegDual
2     int e = no << 1, d = e | 1;
3
4     lazy[e].compose(lazy[no]);
5     lazy[d].compose(lazy[no]);
6
7     lazy[no] = TAG();
8 }
```

Seg Tag

Como a Seg Lazy Dual não guarda valores parciais nos nós, não precisamos de um Node.

Por conta disso, a TAG ganha mais uma responsabilidade:

- Aplicar o update em um valor pontual.
- Guardar o update pendente
- Saber se unir com outra tag

```
1 struct Tag { // SegLazy
2     ll add = 0;
3
4     inline void compose(const Tag& t) {
5         add += t.add;
6     }
7 };
```

Seg Tag



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Como a Seg Lazy Dual não guarda valores parciais nos nós, não precisamos de um Node.

Por conta disso, a TAG ganha mais uma responsabilidade:

- Aplicar o update em um valor pontual.
- Guardar o update pendente
- Saber se unir com outra tag

```
1 struct Tag { // SegDual
2     ll add = 0;
3
4     inline void compose(const Tag& t) {
5         add += t.add;
6     }
7
8     void apply(ll& val) { val += t.add; }
9 };
```

Seg Tag



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Como a Seg Lazy Dual não guarda valores parciais nos nós, não precisamos de um Node.

Por conta disso, a TAG ganha mais uma responsabilidade:

- Aplicar o update em um valor pontual.
- Guardar o update pendente
- Saber se unir com outra tag

```
1 struct Tag { // SegDual
2     ll add = 0;
3
4     inline void compose(const Tag& t) {
5         add += t.add;
6     }
7
8     void apply(ll& val) { val += t.add; }
9 };
```

Seg Tag



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Como a Seg Lazy Dual não guarda valores parciais nos nós, não precisamos de um Node.

Por conta disso, a TAG ganha mais uma responsabilidade:

- Aplicar o update em um valor pontual.
- Guardar o update pendente
- Saber se unir com outra tag

```
1 struct Tag { // SegDual
2     ll add = 0;
3
4     inline void compose(const Tag& t) {
5         add += t.add;
6     }
7
8     void apply(ll& val) { val += t.add; }
9 };
```

Seg Tag



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Como a Seg Lazy Dual não guarda valores parciais nos nós, não precisamos de um Node.

Por conta disso, a TAG ganha mais uma responsabilidade:

- Aplicar o update em um valor pontual.
- Guardar o update pendente
- Saber se unir com outra tag

```
1 struct Tag { // SegDual
2     ll add = 0;
3
4     inline void compose(const Tag& t) {
5         add += t.add;
6     }
7
8     void apply(ll& val) { val += t.add; }
9 };
```

Lazy Vs Dual

Problema: https://judge.yosupo.jp/problem/range_affine_point_get

Lazy Dual:

ID	Data	Problema	Lang	User	Status	Tempo	Memória
350115	2/3/2026	Range Affine Point Get	C++23	lgf	AC ✓	316 ms	39.48 MiB

Lazy Comum:

ID	Data	Problema	Lang	User	Status	Tempo	Memória
350117	2/3/2026	Range Affine Point Get	C++23	lgf	AC ✓	585 ms	50.76 MiB

Resultados:

- Diferença de desempenho: aproximadamente -45% ⚡
- Diferença de Memória: aproximadamente -22% 📉

Dúvidas?

Modificando uma SegTree

Comum, Lazy ou Dual

Seg Lazy Dual

Podemos fazer Seg Trees de quaisquer operações que respeitem algumas propriedades.

Para as segs sem lazy propagation, a operação deve:

- Ser Associativa $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
- Ter elemento neutro $(a \cdot \theta) = a$

Já as segs com lazy, precisam de mais 2 propriedades:

- Distributividade (ou Composição)
- Composição de Lazy Updates

A operação de atualização deve ser capaz de ser aplicada ao resultado de um nó sem necessariamente percorrer todos os seus filhos. Isto é, a operação da TAG sobre o NODE deve ser distributiva em relação à operação de merge do NODE

Soma em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Somar x em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
8      void inline apply(const Tag& t, int l, int r) { val += t.add * (r - l + 1); }
9  };
10
11 struct Tag {
12     ll add = 0; // Elemento Neutro da Soma
13
14     void compose(const Tag& t) { add += t.add; }
15 }
```

Soma em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Somar x em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
8      void inline apply(const Tag& t, int l, int r) { val += t.add * (r - l + 1); }
9  };
10
11 struct Tag {
12     ll add = 0; // Elemento Neutro da Soma
13
14     void compose(const Tag& t) { add += t.add; }
15 }
```

Soma em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Somar x em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
8      void inline apply(const Tag& t, int l, int r) { val += t.add * (r - l + 1); }
9  };
10
11 struct Tag {
12     ll add = 0; // Elemento Neutro da Soma
13
14     void compose(const Tag& t) { add += t.add; }
15 }
```

Soma em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Somar x em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
8      void inline apply(const Tag& t, int l, int r) { val += t.add * (r - l + 1); }
9  };
10
11 struct Tag {
12     ll add = 0; // Elemento Neutro da Soma
13
14     void compose(const Tag& t) { add += t.add; }
15 }
```

Soma em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Somar x em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
8      void inline apply(const Tag& t, int l, int r) { val += t.add * (r - l + 1); }
9  };
10
11 struct Tag {
12     ll add = 0; // Elemento Neutro da Soma
13
14     void compose(const Tag& t) { add += t.add; }
15 }
```

Soma em Range

Update: Somar x em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
8      void inline apply(const Tag& t, int l, int r) { val += t.add * (r - l + 1); }
9  };
10
11 struct Tag {
12     ll add = 0; // Elemento Neutro da Soma
13
14     void compose(const Tag& t) { add += t.add; }
15 }
```


Soma em Range

Update: Somar x em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
8      void inline apply(const Tag& t, int l, int r) { val += t.add * (r - l + 1); }
9  };
10
11 struct Tag {
12     ll add = 0; // Elemento Neutro da Soma
13
14     void compose(const Tag& t) { add += t.add; }
15 }
```

Soma em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Somar x em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
8      void inline apply(const Tag& t, int l, int r) { val += t.add * (r - l + 1); }
9  };
10
11 struct Tag {
12     ll add = 0; // Elemento Neutro da Soma
13
14     void compose(const Tag& t) { add += t.add; }
15 }
```

Soma em Range

Update: Somar x em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
8      void inline apply(const Tag& t, int l, int r) { val += t.add * (r - l + 1); }
9  };
10
11 struct Tag {
12     ll add = 0; // Elemento Neutro da Soma
13
14     void compose(const Tag& t) { add += t.add; }
15 }
```

Set em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Setar x para todo elemento em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  const int NO_UPDATE = -1e9;
2
3  struct Node {
4      ll val = 0; // Elemento Neutro da Soma
5
6      Node() {}
7      Node(ll v) : val(v) {}
8
9      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
10     void inline apply(const Tag& t, int l, int r) { if (t.set != NO_UPDATE) val = t.set * (r - l + 1); }
11 };
12
13 struct Tag {
14     ll set = NO_UPDATE; // Elemento Neutro do set
15
16     void compose(const Tag& t) { if (t.set != NO_UPDATE) set = t.set; }
17 };
```

Set em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Setar x para todo elemento em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  const int NO_UPDATE = -1e9;
2
3  struct Node {
4      ll val = 0; // Elemento Neutro da Soma
5
6      Node() {}
7      Node(ll v) : val(v) {}
8
9      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
10     void inline apply(const Tag& t, int l, int r) { if (t.set != NO_UPDATE) val = t.set * (r - l + 1); }
11 };
12
13 struct Tag {
14     ll set = NO_UPDATE; // Elemento Neutro do set
15
16     void compose(const Tag& t) { if (t.set != NO_UPDATE) set = t.set; }
17 };
18
```

Set em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Setar x para todo elemento em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  const int NO_UPDATE = -1e9;
2
3  struct Node {
4      ll val = 0; // Elemento Neutro da Soma
5
6      Node() {}
7      Node(ll v) : val(v) {}
8
9      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
10     void inline apply(const Tag& t, int l, int r) { if (t.set != NO_UPDATE) val = t.set * (r - l + 1); }
11 };
12
13 struct Tag {
14     ll set = NO_UPDATE; // Elemento Neutro do set
15
16     void compose(const Tag& t) { if (t.set != NO_UPDATE) set = t.set; }
17 };
```

Set em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Setar x para todo elemento em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  const int NO_UPDATE = -1e9;
2
3  struct Node {
4      ll val = 0; // Elemento Neutro da Soma
5
6      Node() {}
7      Node(ll v) : val(v) {}
8
9      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
10     void inline apply(const Tag& t, int l, int r) { if (t.set != NO_UPDATE) val = t.set * (r - l + 1); }
11 };
12
13 struct Tag {
14     ll set = NO_UPDATE; // Elemento Neutro do set
15
16     void compose(const Tag& t) { if (t.set != NO_UPDATE) set = t.set; }
17 };
18
```

Set em Range

Update: Setar x para todo elemento em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  const int NO_UPDATE = -1e9;
2
3  struct Node {
4      ll val = 0; // Elemento Neutro da Soma
5
6      Node() {}
7      Node(ll v) : val(v) {}
8
9      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
10     void inline apply(const Tag& t, int l, int r) { if (t.set != NO_UPDATE) val = t.set * (r - l + 1); }
11 };
12
13 struct Tag {
14     ll set = NO_UPDATE; // Elemento Neutro do set
15
16     void compose(const Tag& t) { if (t.set != NO_UPDATE) set = t.set; }
17 };
```


Set em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Setar x para todo elemento em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  const int NO_UPDATE = -1e9;
2
3  struct Node {
4      ll val = 0; // Elemento Neutro da Soma
5
6      Node() {}
7      Node(ll v) : val(v) {}
8
9      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
10     void inline apply(const Tag& t, int l, int r) { if (t.set != NO_UPDATE) val = t.set * (r - l + 1); }
11 };
12
13 struct Tag {
14     ll set = NO_UPDATE; // Elemento Neutro do set
15
16     void compose(const Tag& t) { if (t.set != NO_UPDATE) set = t.set; }
17 };
```

Set em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Setar x para todo elemento em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  const int NO_UPDATE = -1e9;
2
3  struct Node {
4      ll val = 0; // Elemento Neutro da Soma
5
6      Node() {}
7      Node(ll v) : val(v) {}
8
9      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
10     void inline apply(const Tag& t, int l, int r) { if (t.set != NO_UPDATE) val = t.set * (r - l + 1); }
11 };
12
13 struct Tag {
14     ll set = NO_UPDATE; // Elemento Neutro do set
15
16     void compose(const Tag& t) { if (t.set != NO_UPDATE) set = t.set; }
17 };
```

Set em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Setar x para todo elemento em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  const int NO_UPDATE = -1e9;
2
3  struct Node {
4      ll val = 0; // Elemento Neutro da Soma
5
6      Node() {}
7      Node(ll v) : val(v) {}
8
9      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
10     void inline apply(const Tag& t, int l, int r) { if (t.set != NO_UPDATE) val = t.set * (r - l + 1); }
11 };
12
13 struct Tag {
14     ll set = NO_UPDATE; // Elemento Neutro do set
15
16     void compose(const Tag& t) { if (t.set != NO_UPDATE) set = t.set; }
17 };
```

Set em Range



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Update: Setar x para todo elemento em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  const int NO_UPDATE = -1e9;
2
3  struct Node {
4      ll val = 0; // Elemento Neutro da Soma
5
6      Node() {}
7      Node(ll v) : val(v) {}
8
9      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
10     void inline apply(const Tag& t, int l, int r) { if (t.set != NO_UPDATE) val = t.set * (r - l + 1); }
11 };
12
13 struct Tag {
14     ll set = NO_UPDATE; // Elemento Neutro do set
15
16     void compose(const Tag& t) { if (t.set != NO_UPDATE) set = t.set; }
17 };
```

Set em Range

Update: Setar x para todo elemento em $[L, R]$ | **Query:** Soma de $[L, R]$

```
1  const int NO_UPDATE = -1e9;
2
3  struct Node {
4      ll val = 0; // Elemento Neutro da Soma
5
6      Node() {}
7      Node(ll v) : val(v) {}
8
9      static inline Node merge(const Node& l, const Node& r) { return Node(l.val + r.val); }
10     void inline apply(const Tag& t, int l, int r) { if (t.set != NO_UPDATE) val = t.set * (r - l + 1); }
11 };
12
13 struct Tag {
14     ll set = NO_UPDATE; // Elemento Neutro do set
15
16     void compose(const Tag& t) { if (t.set != NO_UPDATE) set = t.set; }
17 };
```

Soma e produto

Update: Multiplicar por M e Somar A em $[L, R]$ | **Query:** Soma de $[L, R]$ (Com módulo)

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node((l.val + r.val) % MOD); }
8      void inline apply(const Tag& t, int l, int r) {
9          val = (val * t.mul % MOD + t.add * (r - l + 1) % MOD) % MOD;
10     }
11 };
12
13 struct Tag {
14     ll add = 0, mul = 1; // Neutros da Soma e do Produto
15
16     void compose(const Tag& t) {
17         add = (add * t.mul % MOD + t.add) % MOD;
18         mul = (mul * t.mul) % MOD;
19     }
20 };
```

Soma e produto

Update: Multiplicar por M e Somar A em $[L, R]$ | **Query:** Soma de $[L, R]$ (Com módulo)

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node((l.val + r.val) % MOD); }
8      void inline apply(const Tag& t, int l, int r) {
9          val = (val * t.mul % MOD + t.add * (r - l + 1) % MOD) % MOD;
10     }
11 };
12
13 struct Tag {
14     ll add = 0, mul = 1; // Neutros da Soma e do Produto
15
16     void compose(const Tag& t) {
17         add = (add * t.mul % MOD + t.add) % MOD;
18         mul = (mul * t.mul) % MOD;
19     }
20 };
```

Soma e produto

Update: Multiplicar por M e Somar A em $[L, R]$ | **Query:** Soma de $[L, R]$ (Com módulo)

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node((l.val + r.val) % MOD); }
8      void inline apply(const Tag& t, int l, int r) {
9          val = (val * t.mul % MOD + t.add * (r - l + 1) % MOD) % MOD;
10     }
11 };
12
13 struct Tag {
14     ll add = 0, mul = 1; // Neutros da Soma e do Produto
15
16     void compose(const Tag& t) {
17         add = (add * t.mul % MOD + t.add) % MOD;
18         mul = (mul * t.mul) % MOD;
19     }
20 };
```


Soma e produto



Update: Multiplicar por M e Somar A em $[L, R]$ | **Query:** Soma de $[L, R]$ (Com módulo)

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node((l.val + r.val) % MOD); }
8      void inline apply(const Tag& t, int l, int r) {
9          val = (val * t.mul % MOD + t.add * (r - l + 1) % MOD) % MOD;
10     }
11 };
12
13 struct Tag {
14     ll add = 0, mul = 1; // Neutros da Soma e do Produto
15
16     void compose(const Tag& t) {
17         add = (add * t.mul % MOD + t.add) % MOD;
18         mul = (mul * t.mul) % MOD;
19     }
20 };
```

Soma e produto

Update: Multiplicar por M e Somar A em $[L, R]$ | **Query:** Soma de $[L, R]$ (Com módulo)

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node((l.val + r.val) % MOD); }
8      void inline apply(const Tag& t, int l, int r) {
9          val = (val * t.mul % MOD + t.add * (r - l + 1) % MOD) % MOD;
10     }
11 };
12
13 struct Tag {
14     ll add = 0, mul = 1; // Neutros da Soma e do Produto
15
16     void compose(const Tag& t) {
17         add = (add * t.mul % MOD + t.add) % MOD;
18         mul = (mul * t.mul) % MOD;
19     }
20 };
```

Soma e produto

Update: Multiplicar por M e Somar A em $[L, R]$ | **Query:** Soma de $[L, R]$ (Com módulo)

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node((l.val + r.val) % MOD); }
8      void inline apply(const Tag& t, int l, int r) {
9          val = (val * t.mul % MOD + t.add * (r - l + 1) % MOD) % MOD;
10     }
11 };
12
13 struct Tag {
14     ll add = 0, mul = 1; // Neutros da Soma e do Produto
15
16     void compose(const Tag& t) {
17         add = (add * t.mul % MOD + t.add) % MOD;
18         mul = (mul * t.mul) % MOD;
19     }
20 };
```

Soma e produto

Update: Multiplicar por M e Somar A em $[L, R]$ | **Query:** Soma de $[L, R]$ (Com módulo)

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node((l.val + r.val) % MOD); }
8      void inline apply(const Tag& t, int l, int r) {
9          val = (val * t.mul % MOD + t.add * (r - l + 1) % MOD) % MOD;
10     }
11 };
12
13 struct Tag {
14     ll add = 0, mul = 1; // Neutros da Soma e do Produto
15
16     void compose(const Tag& t) {
17         add = (add * t.mul % MOD + t.add) % MOD;
18         mul = (mul * t.mul) % MOD;
19     }
20 };
```

Soma e produto

Update: Multiplicar por M e Somar A em $[L, R]$ | **Query:** Soma de $[L, R]$ (Com módulo)

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node((l.val + r.val) % MOD); }
8      void inline apply(const Tag& t, int l, int r) {
9          val = (val * t.mul % MOD + t.add * (r - l + 1) % MOD) % MOD;
10     }
11 };
12
13 struct Tag {
14     ll add = 0, mul = 1; // Neutros da Soma e do Produto
15
16     void compose(const Tag& t) {
17         add = (add * t.mul % MOD + t.add) % MOD;
18         mul = (mul * t.mul) % MOD;
19     }
20 };
```

Soma e produto



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



Maratona CIn

Update: Multiplicar por M e Somar A em $[L, R]$ | **Query:** Soma de $[L, R]$ (Com módulo)

```
1  struct Node {
2      ll val = 0; // Elemento Neutro da Soma
3
4      Node() {}
5      Node(ll v) : val(v) {}
6
7      static inline Node merge(const Node& l, const Node& r) { return Node((l.val + r.val) % MOD); }
8      void inline apply(const Tag& t, int l, int r) {
9          val = (val * t.mul % MOD + t.add * (r - l + 1) % MOD) % MOD;
10     }
11 };
12
13 struct Tag {
14     ll add = 0, mul = 1; // Neutros da Soma e do Produto
15
16     void compose(const Tag& t) {
17         add = (add * t.mul % MOD + t.add) % MOD;
18         mul = (mul * t.mul) % MOD;
19     }
20 };
```

Outras Operações

```
1 struct FlipTag {
2     bool flip = false;
3
4     void compose(const FlipTag& t) { flip ^= t.flip; }
5 };
```

```
1 ll P[MANX]; // Potências da base pré-calculadas
2
3 struct HNode { // HashNode
4     ll hash = 0; int len = 0;
5
6     HNode() : hash(0), len(0) {}
7     HNode(char c) : hash(c-'a'+1), len(1) {}
8
9     static inline HNode merge(const HNode& l, const HNode& r) {
10         HNode res;
11         res.hash = (l.hash * P[r.len] + r.hash) % MOD;
12         res.len = l.len + r.len;
13         return res;
14     }
15 };
```

```
1 struct SANode { //SetAddNode
2     ll val = 0; SANode(ll val = 0) : val(val) {}
3
4     static inline SANode merge(const SANode& l, const SANode& r) {
5         return SANode(l.val + r.val);
6     }
7
8     void inline apply(const SetAddTag& t, int l, int r) {
9         if (t.set != NO_UPDATE) val = t.set * (r - l + 1);
10        val += t.add * (r - l + 1);
11    }
12 };
13
14 struct SetAddTag {
15     ll set = NO_UPDATE, add = 0;
16
17     void compose(const SetAddTag& t) {
18         if (t.set != NO_UPDATE) set = t.set, add = 0;
19         else {
20             if (set != NO_UPDATE) set += t.add;
21             else add += t.add;
22         }
23     }
24 };
```

Dúvidas?

SegTree Esparsa/Dinâmica

Lidando com coordenadas altas

SegTree Esparsa



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Por vezes, queremos fazer queries sob intervalos grande, como $[0, 10^9]$, mas o número de updates (q) é pequeno (ex: 10^5). Como lidamos com isso se o espaço da Seg é $\mathcal{O}(4 \times N)$?

- **Limitação:** Um array de 4×10^9 é impossível de alocar.

Soluções:

- **Compressão de Coordenadas:** Ótima se você tiver todos os updates offline.
- **SegTree Dinâmica:** Necessária se os updates forem online. (Ou se você não quiser lidar com compressão)

SegTree Esparsa

Para fazer uma árvore esparsa (ou dinâmica) podemos usar uma estrutura de ponteiros para criar os nós sob demanda, somente quando uma query ou update tentar acessá-lo. Dessa forma, cada nó da árvore tem um ponteiro para quem é seu filho à esquerda e à direita.

Para evitar o *overhead* de ponteiros e de alocação de memória na *heap* (lento), podemos guardar a estrutura inteira em um `vector<InternalNode>` e usar índices inteiros.

```
1      struct InternalNode {
2          NODE node;
3          TAG tag;
4          InternalNode *l, *r;
5          InternalNode() : node(NODE()), tag(TAG()),
6                          l(nullptr), r(nullptr) {}
7
8          void apply(const TAG& t, ll l, ll r) {
9              node.apply(t, l, r);
10             tag.compose(t);
11         }
12     };
```

SegTree Esparsa

Para fazer uma árvore esparsa (ou dinâmica) podemos usar uma estrutura de ponteiros para criar os nós sob demanda, somente quando uma query ou update tentar acessá-lo. Dessa forma, cada nó da árvore tem um ponteiro para quem é seu filho à esquerda e à direita.

Para evitar o *overhead* de ponteiros e de alocação de memória na *heap* (lento), podemos guardar a estrutura inteira em um `vector<InternalNode>` e usar índices inteiros.

```
1  template<typename NODE, typename TAG>
2  struct DynamicSegTree {
3      struct InternalNode {
4          NODE node;
5          TAG tag;
6          int l, r;
7          InternalNode() : node(NODE()), tag(TAG()),
8                          l(-1), r(-1) {}
9
10         void apply(const TAG& t, ll l, ll r) {
11             node.apply(t, l, r);
12             tag.compose(t);
13         }
14     };
15
16     ll L, R;
17     vector<InternalNode> seg;
18
19     ...
20 };
```

SegTree Esparsa



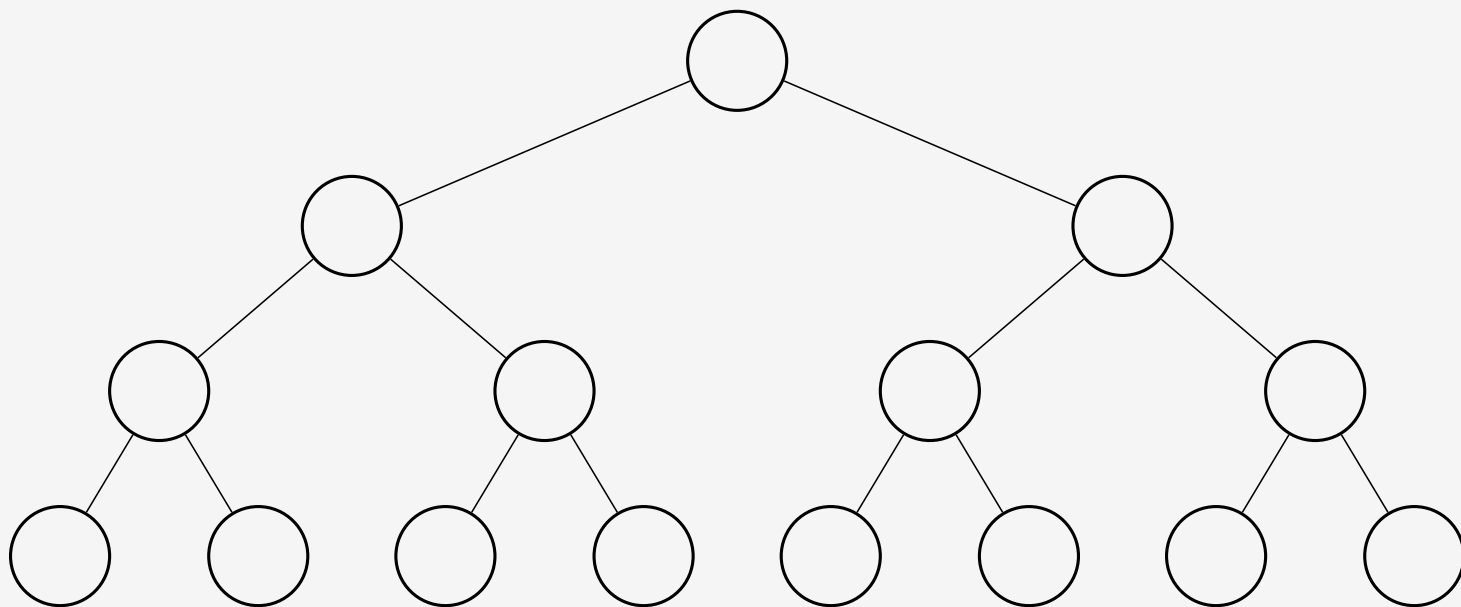
Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



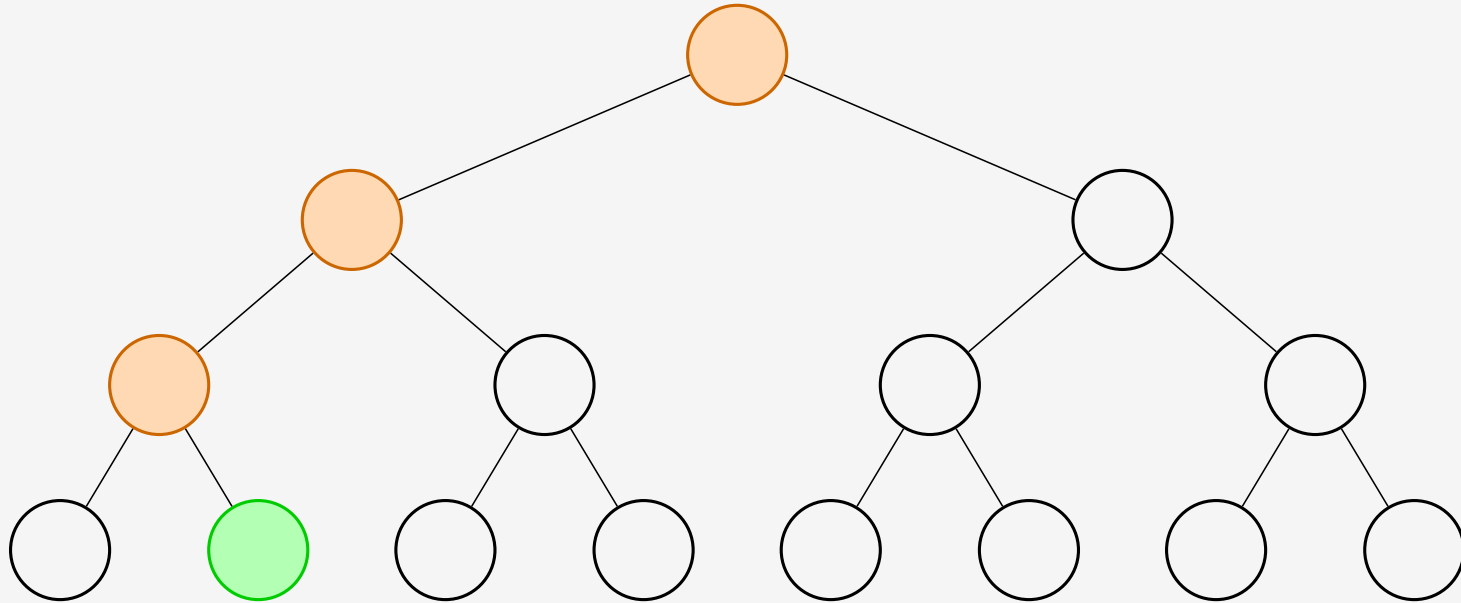
MaratonaCIn



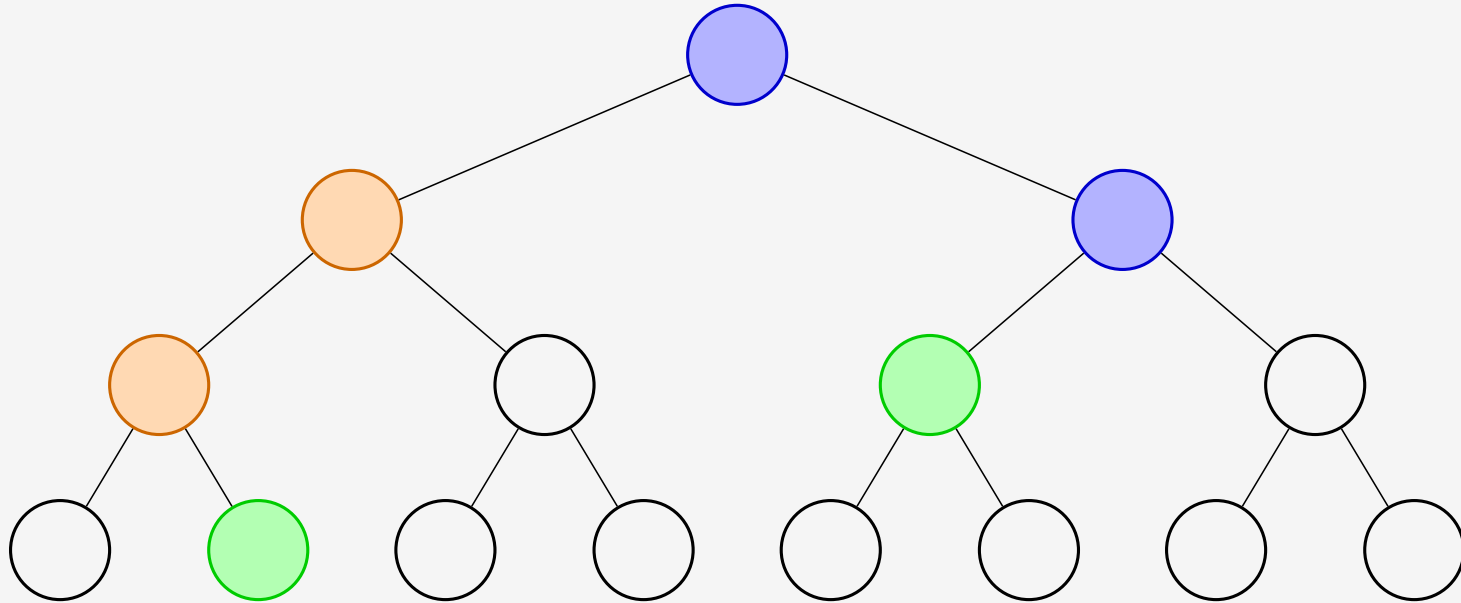
Centro de Informática
 UFPE

 UNIVERSIDADE
 FEDERAL
 DE PERNAMBUCO

Maratona CIn



SegTree Esparsa



SegTree Esparsa



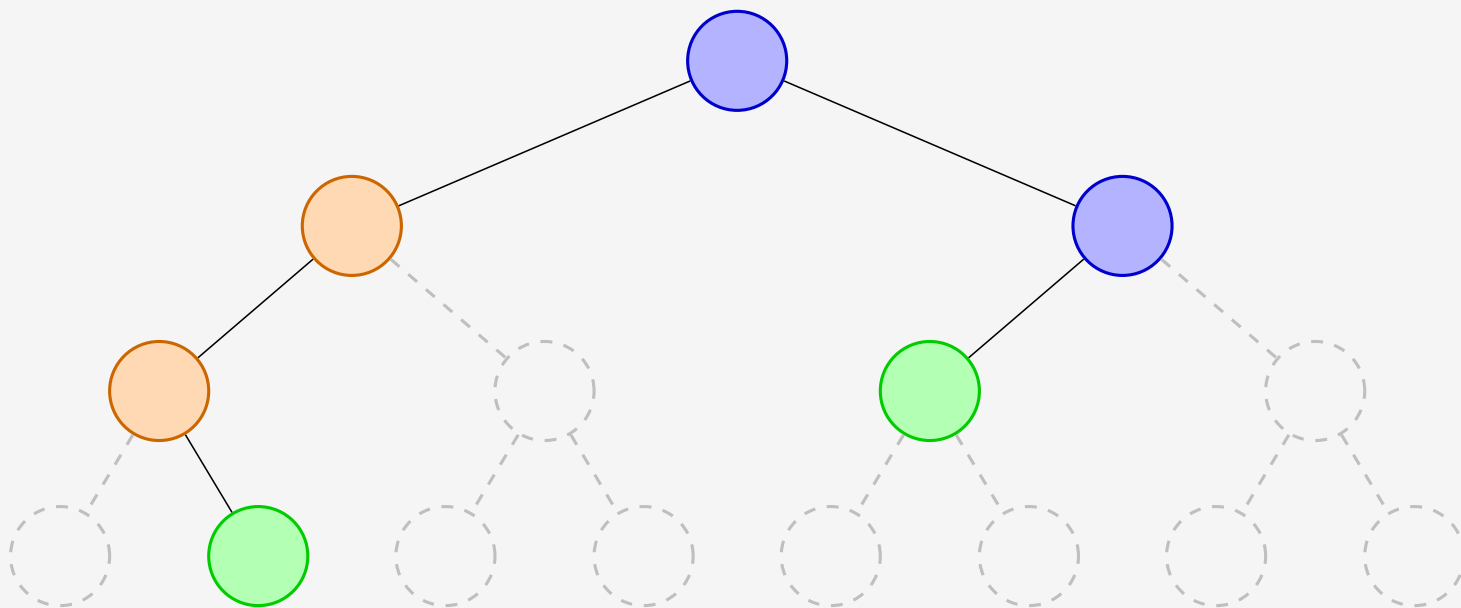
Centro de
Informática
UFPE



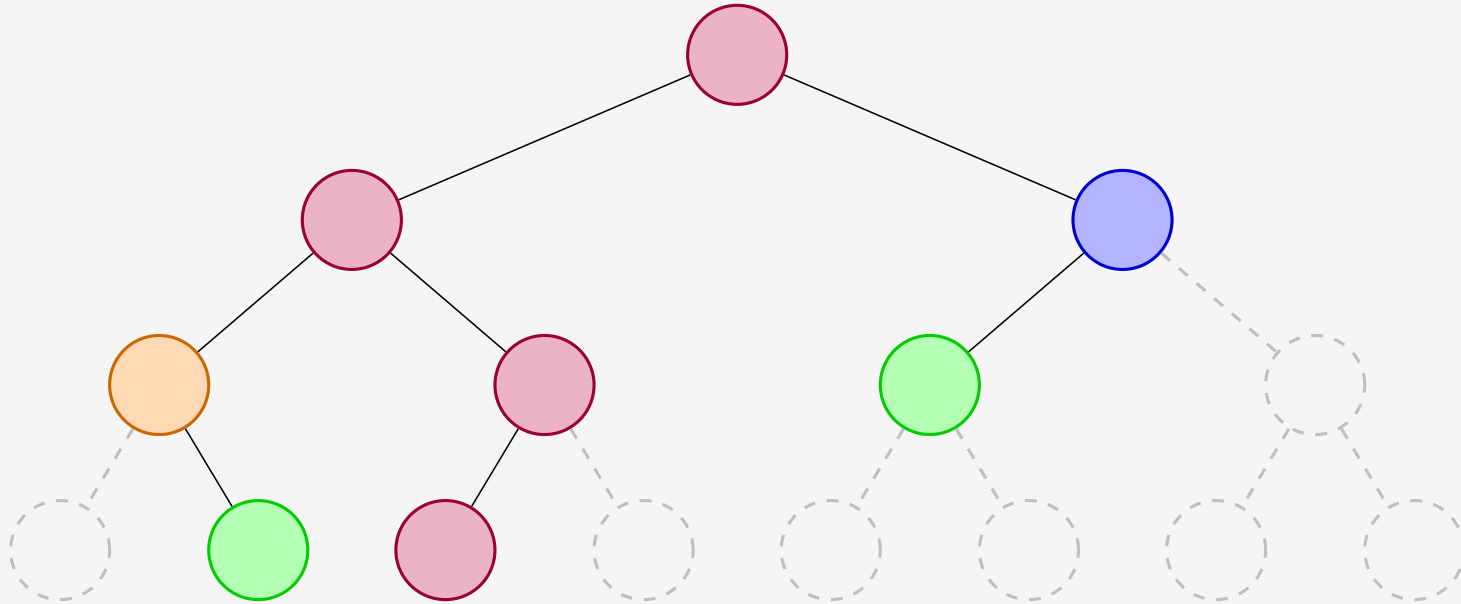
UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



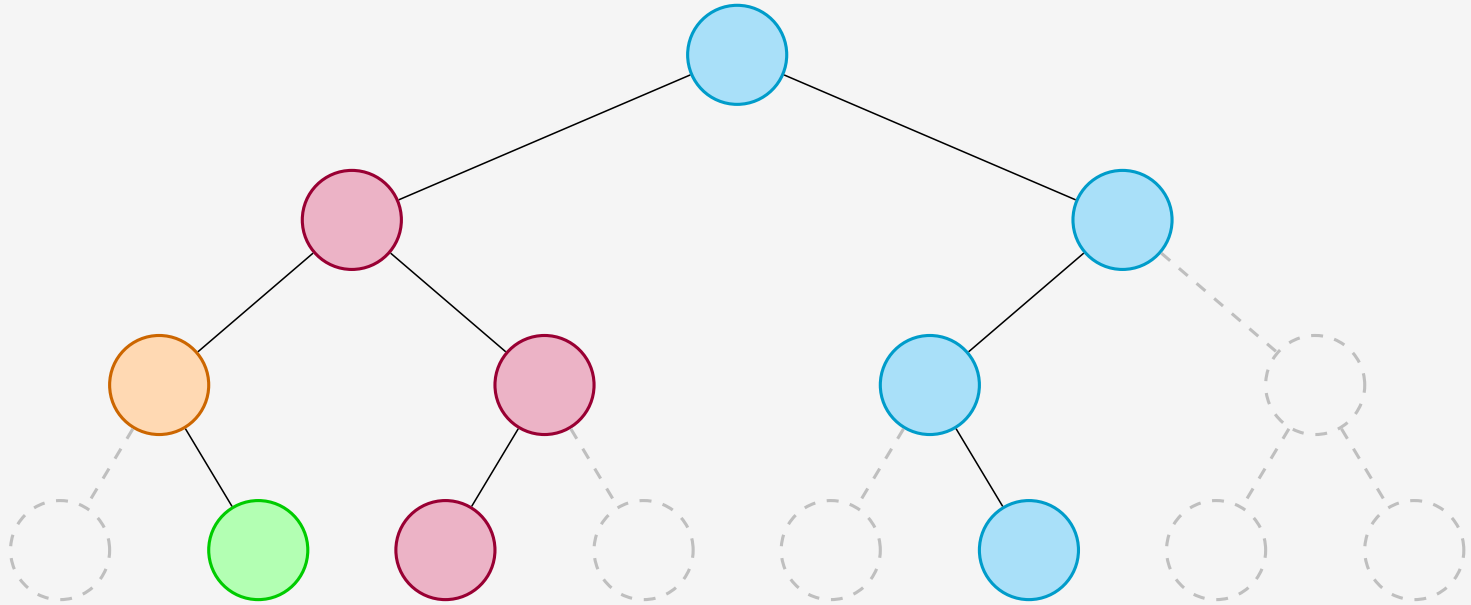
MaratonaCIn



SegTree Esparsa



SegTree Esparsa



```
1  template<typename NODE, typename TAG>
2  struct LazySegmentTree {
3      int N;
4      vector<NODE> seg;
5      vector<TAG> lazy;
6
7      LazySegmentTree(int n) : N(n), seg(4 * n), lazy(4 * n) {}
8
9      LazySegmentTree(const vector<int>& v)
10         : N(v.size()), seg(4 * v.size()), lazy(4 * v.size()) {
11         build(1, 0, N - 1, v);
12     }
13
14     void build(int no, int l, int r, const vector<int>& v) {
15         if (l == r) {
16             seg[no] = NODE(v[l]);
17             return;
18         }
19         int m = (l + r) >> 1;
20         build(no << 1, l, m, v);
21         build((no << 1) | 1, m + 1, r, v);
22         seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
23     }
```

```
1  template<typename NODE, typename TAG>
2  struct DynamicSegTree {
3      struct InternalNode {
4          NODE node;
5          TAG tag;
6          int l, r;
7          InternalNode() : node(NODE()), tag(TAG()), l(-1), r(-1) {}
8
9          void apply(const TAG& t, ll l, ll r) {
10              node.apply(t, l, r);
11              tag.compose(t);
12          }
13      };
14
15      ll L, R;
16      vector<InternalNode> seg;
17
18      DynamicSegTree(ll l, ll r) : L(l), R(r) {
19          seg.reserve(4e6); seg.emplace_back();
20      }
```

```
1  template<typename NODE, typename TAG>
2  struct DynamicSegTree {
3      struct InternalNode {
4          NODE node;
5          TAG tag;
6          int l, r;
7          InternalNode() : node(NODE()), tag(TAG()), l(-1), r(-1) {}
8
9          void apply(const TAG& t, ll l, ll r) {
10              node.apply(t, l, r);
11              tag.compose(t);
12          }
13      };
14
15      ll L, R;
16      vector<InternalNode> seg;
17
18      DynamicSegTree(ll l, ll r) : L(l), R(r) {
19          seg.reserve(4e6); seg.emplace_back();
20      }
```

```
1  template<typename NODE, typename TAG>
2  struct DynamicSegTree {
3      struct InternalNode {
4          NODE node;
5          TAG tag;
6          int l, r;
7          InternalNode() : node(NODE()), tag(TAG()), l(-1), r(-1) {}
8
9          void apply(const TAG& t, ll l, ll r) {
10              node.apply(t, l, r);
11              tag.compose(t);
12          }
13     };
14
15     ll L, R;
16     vector<InternalNode> seg;
17
18     DynamicSegTree(ll l, ll r) : L(l), R(r) {
19         seg.reserve(4e6); seg.emplace_back();
20     }
```

```
1  template<typename NODE, typename TAG>
2  struct DynamicSegTree {
3      struct InternalNode {
4          NODE node;
5          TAG tag;
6          int l, r;
7          InternalNode() : node(NODE()), tag(TAG()), l(-1), r(-1) {}
8
9          void apply(const TAG& t, ll l, ll r) {
10              node.apply(t, l, r);
11              tag.compose(t);
12          }
13     };
14
15     ll L, R;
16     vector<InternalNode> seg;
17
18     DynamicSegTree(ll l, ll r) : L(l), R(r) {
19         seg.reserve(4e6); seg.emplace_back();
20     }
```

```
1 void update(int no, int l, int r, int a, int b, const TAG& v) { // SegLazy
2     if (b < l || r < a) return;
3     if (a ≤ l && r ≤ b) {
4         seg[no].apply(v, l, r);
5         lazy[no].compose(v);
6         return;
7     }
8     push(no, l, r);
9     int m = (l + r) >> 1;
10    update(no << 1, l, m, a, b, v);
11    update((no << 1) | 1, m + 1, r, a, b, v);
12    seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
13 }
```



```
1 void update(int v, ll l, ll r, ll ql, ll qr, const TAG& t) { // Seg Esparsa
2     if (ql ≤ l && r ≤ qr) return seg[v].apply(t, l, r);
3
4     push(v, l, r);
5     ll mid = l + (r - l) / 2;
6     if (ql ≤ mid) {
7         if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
8         update(seg[v].l, l, mid, ql, qr, t);
9     }
10    if (qr > mid) {
11        if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
12        update(seg[v].r, mid + 1, r, ql, qr, t);
13    }
14
15    NODE left = (seg[v].l ≠ -1) ? seg[seg[v].l].node : NODE();
16    NODE right = (seg[v].r ≠ -1) ? seg[seg[v].r].node : NODE();
17    seg[v].node = NODE::merge(left, right);
18 }
```

```
1 void update(int v, ll l, ll r, ll ql, ll qr, const TAG& t) { // Seg Esparsa
2     if (ql ≤ l && r ≤ qr) return seg[v].apply(t, l, r);
3
4     push(v, l, r);
5     ll mid = l + (r - l) / 2;
6     if (ql ≤ mid) {
7         if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
8         update(seg[v].l, l, mid, ql, qr, t);
9     }
10    if (qr > mid) {
11        if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
12        update(seg[v].r, mid + 1, r, ql, qr, t);
13    }
14
15    NODE left = (seg[v].l ≠ -1) ? seg[seg[v].l].node : NODE();
16    NODE right = (seg[v].r ≠ -1) ? seg[seg[v].r].node : NODE();
17    seg[v].node = NODE::merge(left, right);
18 }
```

```
1 void update(int v, ll l, ll r, ll ql, ll qr, const TAG& t) { // Seg Esparsa
2     if (ql ≤ l && r ≤ qr) return seg[v].apply(t, l, r);
3
4     push(v, l, r);
5     ll mid = l + (r - l) / 2;
6     if (ql ≤ mid) {
7         if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
8         update(seg[v].l, l, mid, ql, qr, t);
9     }
10    if (qr > mid) {
11        if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
12        update(seg[v].r, mid + 1, r, ql, qr, t);
13    }
14
15    NODE left = (seg[v].l ≠ -1) ? seg[seg[v].l].node : NODE();
16    NODE right = (seg[v].r ≠ -1) ? seg[seg[v].r].node : NODE();
17    seg[v].node = NODE::merge(left, right);
18 }
```

```
1 void update(int v, ll l, ll r, ll ql, ll qr, const TAG& t) { // Seg Esparsa
2     if (ql ≤ l && r ≤ qr) return seg[v].apply(t, l, r);
3
4     push(v, l, r);
5     ll mid = l + (r - l) / 2;
6     if (ql ≤ mid) {
7         if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
8         update(seg[v].l, l, mid, ql, qr, t);
9     }
10    if (qr > mid) {
11        if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
12        update(seg[v].r, mid + 1, r, ql, qr, t);
13    }
14
15    NODE left = (seg[v].l ≠ -1) ? seg[seg[v].l].node : NODE();
16    NODE right = (seg[v].r ≠ -1) ? seg[seg[v].r].node : NODE();
17    seg[v].node = NODE::merge(left, right);
18 }
```

```
1 void update(int v, ll l, ll r, ll ql, ll qr, const TAG& t) { // Seg Esparsa
2     if (ql ≤ l && r ≤ qr) return seg[v].apply(t, l, r);
3
4     push(v, l, r);
5     ll mid = l + (r - l) / 2;
6     if (ql ≤ mid) {
7         if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
8         update(seg[v].l, l, mid, ql, qr, t);
9     }
10    if (qr > mid) {
11        if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
12        update(seg[v].r, mid + 1, r, ql, qr, t);
13    }
14
15    NODE left = (seg[v].l ≠ -1) ? seg[seg[v].l].node : NODE();
16    NODE right = (seg[v].r ≠ -1) ? seg[seg[v].r].node : NODE();
17    seg[v].node = NODE::merge(left, right);
18 }
```

```
1 void update(int v, ll l, ll r, ll ql, ll qr, const TAG& t) { // Seg Esparsa
2     if (ql ≤ l && r ≤ qr) return seg[v].apply(t, l, r);
3
4     push(v, l, r);
5     ll mid = l + (r - l) / 2;
6     if (ql ≤ mid) {
7         if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
8         update(seg[v].l, l, mid, ql, qr, t);
9     }
10    if (qr > mid) {
11        if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
12        update(seg[v].r, mid + 1, r, ql, qr, t);
13    }
14
15    NODE left = (seg[v].l ≠ -1) ? seg[seg[v].l].node : NODE();
16    NODE right = (seg[v].r ≠ -1) ? seg[seg[v].r].node : NODE();
17    seg[v].node = NODE::merge(left, right);
18 }
```

```
1  NODE query(int no, int l, int r, int a, int b) { // SegLazy
2      if (b < l || r < a) return NODE();
3      if (a ≤ l && r ≤ b) return seg[no];
4      push(no, l, r);
5      int m = (l + r) >> 1;
6      return NODE::merge(query(no << 1, l, m, a, b),
7                          query((no << 1) | 1, m + 1, r, a, b));
8  }
9
10 void update(int l, int r, const TAG& v) { update(1, 0, N - 1, l, r, v); }
11 NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
```

```
1  NODE query(int v, ll l, ll r, ll ql, ll qr) { // Seg Esparsa
2      if (v == -1 || ql > r || qr < l) return NODE();
3      if (ql ≤ l && r ≤ qr) return seg[v].node;
4      push(v, l, r);
5      ll mid = l + (r - l) / 2;
6      return NODE::merge(query(seg[v].l, l, mid, ql, qr),
7                          query(seg[v].r, mid + 1, r, ql, qr));
8  }
9
10 void update(ll ql, ll qr, const TAG& t) { update(0, L, R, ql, qr, t); }
11 NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
```



```
1  NODE query(int v, ll l, ll r, ll ql, ll qr) { // Seg Esparsa
2      if (v == -1 || ql > r || qr < l) return NODE();
3      if (ql ≤ l && r ≤ qr) return seg[v].node;
4      push(v, l, r);
5      ll mid = l + (r - l) / 2;
6      return NODE::merge(query(seg[v].l, l, mid, ql, qr),
7                          query(seg[v].r, mid + 1, r, ql, qr));
8  }
9
10 void update(ll ql, ll qr, const TAG& t) { update(0, L, R, ql, qr, t); }
11 NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
```

```
1  NODE query(int v, ll l, ll r, ll ql, ll qr) { // Seg Esparsa
2      if (v == -1 || ql > r || qr < l) return NODE();
3      if (ql ≤ l && r ≤ qr) return seg[v].node;
4      push(v, l, r);
5      ll mid = l + (r - l) / 2;
6      return NODE::merge(query(seg[v].l, l, mid, ql, qr),
7                          query(seg[v].r, mid + 1, r, ql, qr));
8  }
9
10 void update(ll ql, ll qr, const TAG& t) { update(0, L, R, ql, qr, t); }
11 NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
```

```
1  NODE query(int v, ll l, ll r, ll ql, ll qr) { // Seg Esparsa
2      if (v == -1 || ql > r || qr < l) return NODE();
3      if (ql ≤ l && r ≤ qr) return seg[v].node;
4      push(v, l, r);
5      ll mid = l + (r - l) / 2;
6      return NODE::merge(query(seg[v].l, l, mid, ql, qr),
7                          query(seg[v].r, mid + 1, r, ql, qr));
8  }
9
10 void update(ll ql, ll qr, const TAG& t) { update(0, L, R, ql, qr, t); }
11 NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
```

```
1  NODE query(int v, ll l, ll r, ll ql, ll qr) { // Seg Esparsa
2      if (v == -1 || ql > r || qr < l) return NODE();
3      if (ql ≤ l && r ≤ qr) return seg[v].node;
4      push(v, l, r);
5      ll mid = l + (r - l) / 2;
6      return NODE::merge(query(seg[v].l, l, mid, ql, qr),
7                          query(seg[v].r, mid + 1, r, ql, qr));
8  }
9
10 void update(ll ql, ll qr, const TAG& t) { update(0, L, R, ql, qr, t); }
11 NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
```

```
1  NODE query(int v, ll l, ll r, ll ql, ll qr) { // Seg Esparsa
2      if (v == -1 || ql > r || qr < l) return NODE();
3      if (ql ≤ l && r ≤ qr) return seg[v].node;
4      push(v, l, r);
5      ll mid = l + (r - l) / 2;
6      return NODE::merge(query(seg[v].l, l, mid, ql, qr),
7                          query(seg[v].r, mid + 1, r, ql, qr));
8  }
9
10 void update(ll ql, ll qr, const TAG& t) { update(0, L, R, ql, qr, t); }
11 NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
```

```
1 void push(int no, int l, int r) { // SegLazy
2     int m = (l + r) >> 1;
3     int e = no << 1, d = e | 1;
4
5     seg[e].apply(lazy[no], l, m);
6     lazy[e].compose(lazy[no]);
7
8     seg[d].apply(lazy[no], m + 1, r);
9     lazy[d].compose(lazy[no]);
10
11     lazy[no] = TAG();
12 }
```

```
1 void push(int v, ll l, ll r) { // Seg Esparsa
2     if (seg[v].tag == TAG()) return;
3     if (l == r) {
4         seg[v].tag = TAG();
5         return;
6     }
7
8     if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
9     if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
10
11     ll mid = l + (r - l) / 2;
12     seg[seg[v].l].apply(seg[v].tag, l, mid);
13     seg[seg[v].r].apply(seg[v].tag, mid+1, r);
14
15     seg[v].tag = TAG();
16 }
```

```
1 void push(int v, ll l, ll r) { // Seg Esparsa
2     if (seg[v].tag == TAG()) return;
3     if (l == r) {
4         seg[v].tag = TAG();
5         return;
6     }
7
8     if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
9     if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
10
11     ll mid = l + (r - l) / 2;
12     seg[seg[v].l].apply(seg[v].tag, l, mid);
13     seg[seg[v].r].apply(seg[v].tag, mid+1, r);
14
15     seg[v].tag = TAG();
16 }
```



```
1 void push(int v, ll l, ll r) { // Seg Esparsa
2     if (seg[v].tag == TAG()) return;
3     if (l == r) {
4         seg[v].tag = TAG();
5         return;
6     }
7
8     if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
9     if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
10
11     ll mid = l + (r - l) / 2;
12     seg[seg[v].l].apply(seg[v].tag, l, mid);
13     seg[seg[v].r].apply(seg[v].tag, mid+1, r);
14
15     seg[v].tag = TAG();
16 }
```

```
1 void push(int v, ll l, ll r) { // Seg Esparsa
2     if (seg[v].tag == TAG()) return;
3     if (l == r) {
4         seg[v].tag = TAG();
5         return;
6     }
7
8     if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
9     if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
10
11     ll mid = l + (r - l) / 2;
12     seg[seg[v].l].apply(seg[v].tag, l, mid);
13     seg[seg[v].r].apply(seg[v].tag, mid+1, r);
14
15     seg[v].tag = TAG();
16 }
```

```
1 void push(int v, ll l, ll r) { // Seg Esparsa
2     if (seg[v].tag == TAG()) return;
3     if (l == r) {
4         seg[v].tag = TAG();
5         return;
6     }
7
8     if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
9     if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
10
11     ll mid = l + (r - l) / 2;
12     seg[seg[v].l].apply(seg[v].tag, l, mid);
13     seg[seg[v].r].apply(seg[v].tag, mid+1, r);
14
15     seg[v].tag = TAG();
16 }
```

```
1 void push(int v, ll l, ll r) { // Seg Esparsa
2     if (seg[v].tag == TAG()) return;
3     if (l == r) {
4         seg[v].tag = TAG();
5         return;
6     }
7
8     if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
9     if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
10
11     ll mid = l + (r - l) / 2;
12     seg[seg[v].l].apply(seg[v].tag, l, mid);
13     seg[seg[v].r].apply(seg[v].tag, mid+1, r);
14
15     seg[v].tag = TAG();
16 }
```

```
1 void push(int v, ll l, ll r) { // Seg Esparsa
2     if (seg[v].tag == TAG()) return;
3     if (l == r) {
4         seg[v].tag = TAG();
5         return;
6     }
7
8     if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
9     if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
10
11     ll mid = l + (r - l) / 2;
12     seg[seg[v].l].apply(seg[v].tag, l, mid);
13     seg[seg[v].r].apply(seg[v].tag, mid+1, r);
14
15     seg[v].tag = TAG();
16 }
```

Dúvidas?

SegTree Persistente

Voltando no tempo

SegTree Persistente



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Por vezes, queremos realizar queries em uma versão anterior da estrutura de dados.

- Exemplo: "Qual era a soma no range $[L, R]$ logo após o k -ésimo update?"

A Abordagem Ingênua: Clonar a árvore inteira a cada update.

Guardamos um array de SegTrees:
`vector<SegTree> versions;`

O Gargalo:

- Tempo: $\mathcal{O}(N \cdot Q)$ para copiar tudo a cada update.
- Memória: $\mathcal{O}(N \cdot Q)$. Se $N = 10^5$ e $Q = 10^5$, precisamos de 10^{10} posições. Explode a memória!

Path Copying

Um update pontual altera apenas o caminho da raiz até a folha ($\mathcal{O}(\log N)$ nós).

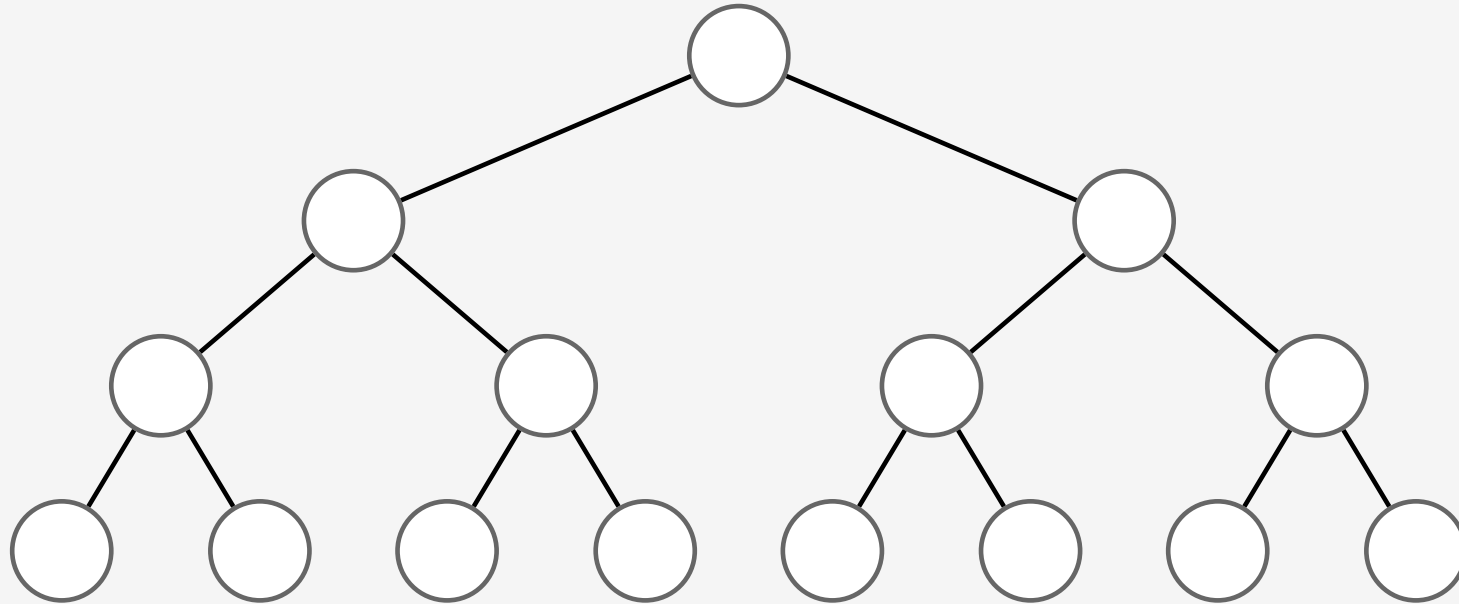
Todo o restante da árvore (as sub-árvores irmãs do caminho) permanece idêntico.

Portanto, em vez de copiar tudo, reaproveitamos os nós que não mudaram.

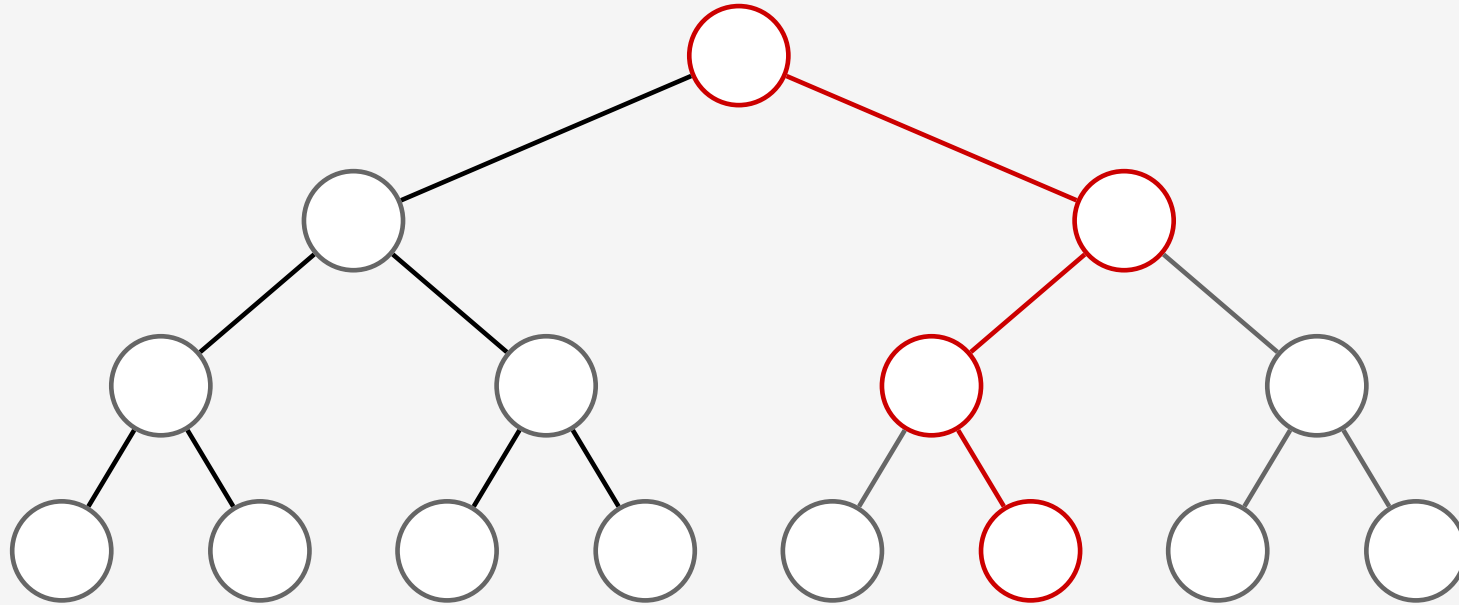
- Criamos apenas os nós novos do caminho alterado.
- Os novos nós apontam para os filhos "velhos" quando o caminho não segue por lá.

É essencialmente uma SegTree Dinâmica onde nunca deletamos nós. Temos um array de raízes: $roots[k]$ aponta para a raiz da árvore no tempo k .

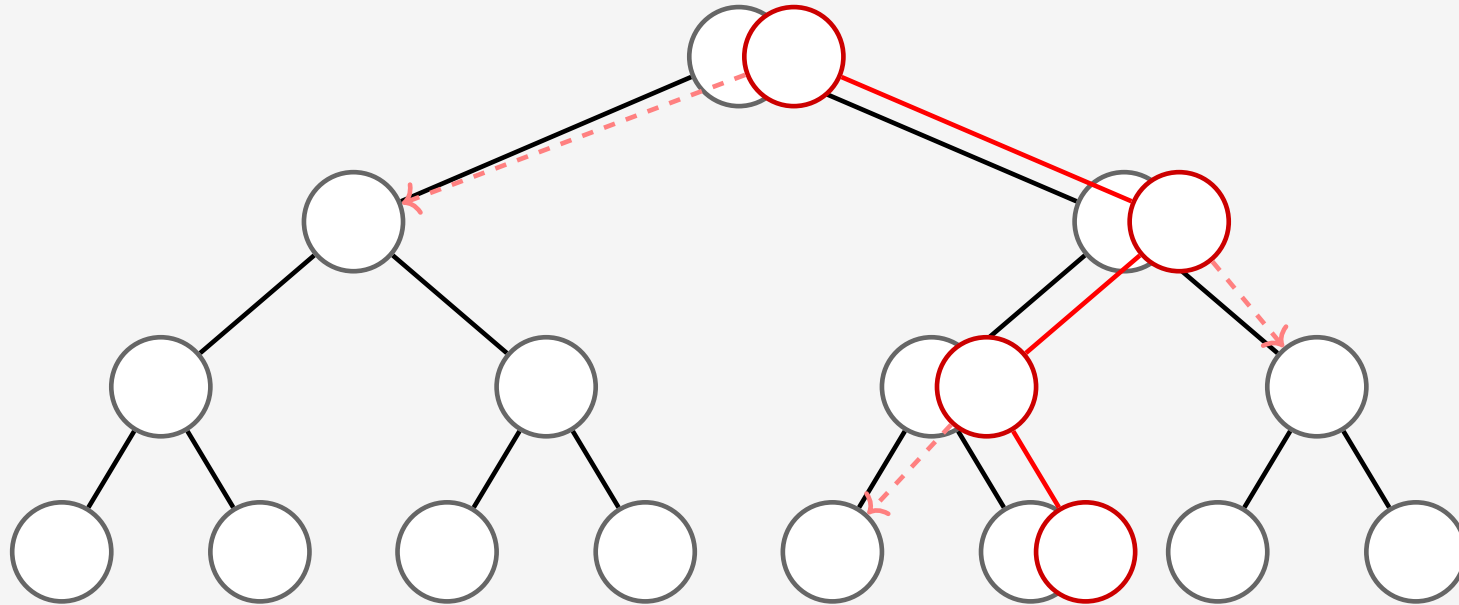
SegTree Persistente



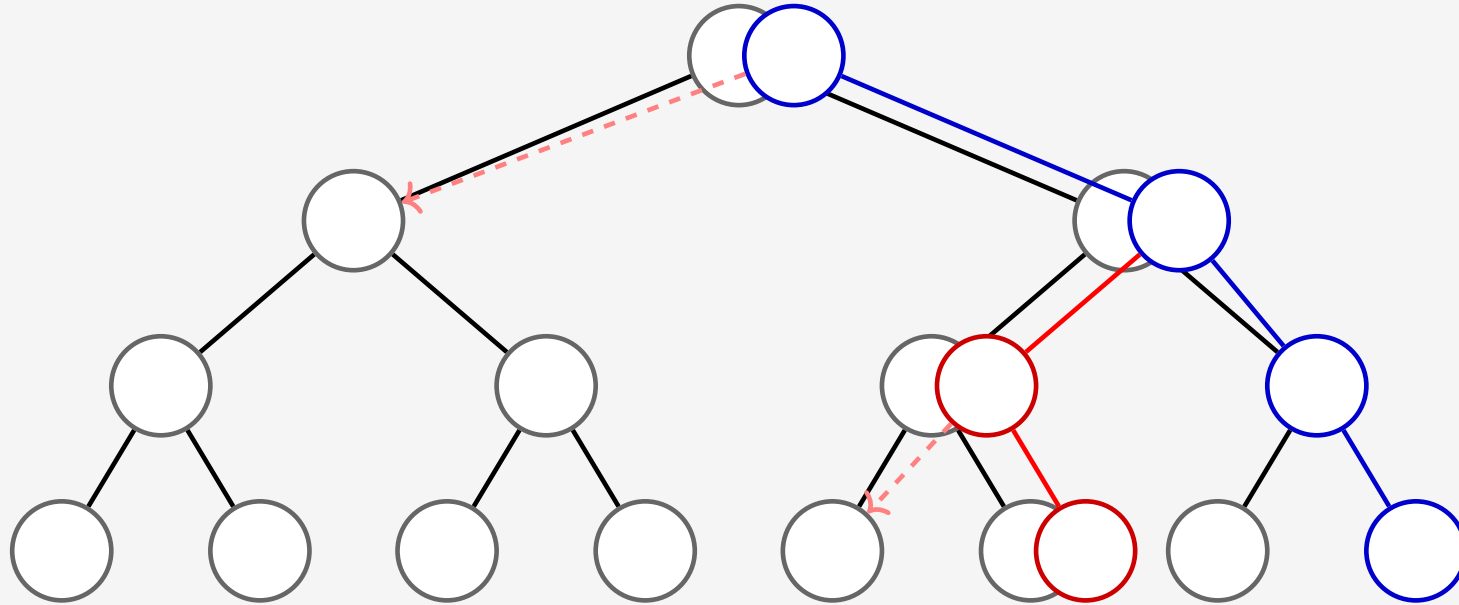
SegTree Persistente



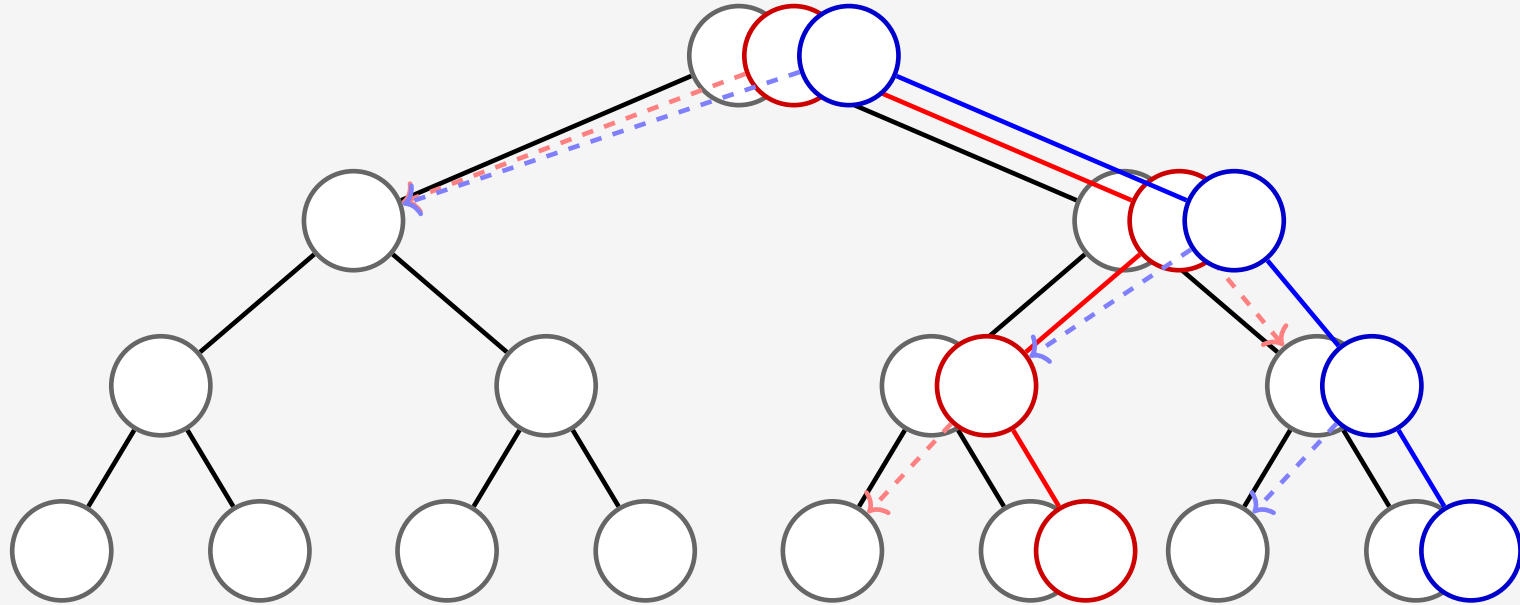
SegTree Persistente



SegTree Persistente



SegTree Persistente



```
1  template<typename NODE, typename TAG>
2  struct DynamicSegTree {
3      struct InternalNode {
4          NODE node;
5          TAG tag;
6          int l, r;
7          InternalNode() : node(NODE()), tag(TAG()), l(-1), r(-1) {}
8
9          void apply(const TAG& t, ll l, ll r) {
10              node.apply(t, l, r);
11              tag.compose(t);
12          }
13     };
14
15     ll L, R;
16     vector<InternalNode> seg;
17
18     DynamicSegTree(ll l, ll r) : L(l), R(r) {
19         seg.reserve(4e6); seg.emplace_back();
20     }
```

```
1  template<typename NODE>
2  struct PersistentSegmentTree {
3      struct InternalNode {
4          NODE data;
5          int l, r;
6          InternalNode() : data(NODE()), l(0), r(0) {}
7      };
8
9      int N;
10     vector<InternalNode> seg;
11     vector<int> roots;
12
13     PersistentSegmentTree(int n) : N(n) {
14         seg.reserve(4e6); seg.emplace_back();
15         roots.push_back(0);
16     }
```



```
1  template<typename NODE>
2  struct PersistentSegmentTree {
3      struct InternalNode {
4          NODE data;
5          int l, r;
6          InternalNode() : data(NODE()), l(0), r(0) {}
7      };
8
9      int N;
10     vector<InternalNode> seg;
11     vector<int> roots;
12
13     PersistentSegmentTree(int n) : N(n) {
14         seg.reserve(4e6); seg.emplace_back();
15         roots.push_back(0);
16     }
```

```
1  template<typename NODE>
2  struct PersistentSegmentTree {
3      struct InternalNode {
4          NODE data;
5          int l, r;
6          InternalNode() : data(NODE()), l(0), r(0) {}
7      };
8
9      int N;
10     vector<InternalNode> seg;
11     vector<int> roots;
12
13     PersistentSegmentTree(int n) : N(n) {
14         seg.reserve(4e6); seg.emplace_back();
15         roots.push_back(0);
16     }
```

```
1  template<typename NODE>
2  struct PersistentSegmentTree {
3      struct InternalNode {
4          NODE data;
5          int l, r;
6          InternalNode() : data(NODE()), l(0), r(0) {}
7      };
8
9      int N;
10     vector<InternalNode> seg;
11     vector<int> roots;
12
13     PersistentSegmentTree(int n) : N(n) {
14         seg.reserve(4e6); seg.emplace_back();
15         roots.push_back(0);
16     }
```

```
1  template<typename NODE>
2  struct PersistentSegmentTree {
3      struct InternalNode {
4          NODE data;
5          int l, r;
6          InternalNode() : data(NODE()), l(0), r(0) {}
7      };
8
9      int N;
10     vector<InternalNode> seg;
11     vector<int> roots;
12
13     PersistentSegmentTree(int n) : N(n) {
14         seg.reserve(4e6); seg.emplace_back();
15         roots.push_back(0);
16     }
```

```
1 void update(int v, ll l, ll r, ll ql, ll qr, const TAG& t) { // Seg Esparsa
2     if (ql ≤ l && r ≤ qr) return seg[v].apply(t, l, r);
3
4     push(v, l, r);
5     ll mid = l + (r - l) / 2;
6     if (ql ≤ mid) {
7         if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
8         update(seg[v].l, l, mid, ql, qr, t);
9     }
10    if (qr > mid) {
11        if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
12        update(seg[v].r, mid + 1, r, ql, qr, t);
13    }
14
15    NODE left = (seg[v].l ≠ -1) ? seg[seg[v].l].node : NODE();
16    NODE right = (seg[v].r ≠ -1) ? seg[seg[v].r].node : NODE();
17    seg[v].node = NODE::merge(left, right);
18 }
```

```
1  int update(int prev_root, int L, int R, int idx, NODE v) { // Seg Persistente
2      int node = seg.size();
3      seg.push_back(prev_root == 0 ? Node() : seg[prev_root]);
4
5      if (L == R) { seg[node].data.apply(v); return node }
6
7      int mid = L + (R - L) / 2;
8
9      if (idx ≤ mid) { // Cuidado com ordem de avaliacao (LHS) pre-C++17
10         int prev_left = (prev_root == 0) ? 0 : seg[prev_root].l;
11         seg[node].l = update(prev_left, L, mid, idx, v);
12     }
13     else { // Cuidado com ordem de avaliacao (LHS) pre-C++17
14         int prev_right = (prev_root == 0) ? 0 : seg[prev_root].r;
15         seg[node].r = update(prev_right, mid + 1, R, idx, v);
16     }
17
18     seg[node].data = NODE::merge(seg[seg[node].l].data, seg[seg[node].r].data);
19     return node;
20 }
```

```
1  int update(int prev_root, int L, int R, int idx, NODE v) { // Seg Persistente
2      int node = seg.size();
3      seg.push_back(prev_root == 0 ? Node() : seg[prev_root]);
4
5      if (L == R) { seg[node].data.apply(v); return node }
6
7      int mid = L + (R - L) / 2;
8
9      if (idx ≤ mid) { // Cuidado com ordem de avaliacao (LHS) pre-C++17
10         int prev_left = (prev_root == 0) ? 0 : seg[prev_root].l;
11         seg[node].l = update(prev_left, L, mid, idx, v);
12     }
13     else { // Cuidado com ordem de avaliacao (LHS) pre-C++17
14         int prev_right = (prev_root == 0) ? 0 : seg[prev_root].r;
15         seg[node].r = update(prev_right, mid + 1, R, idx, v);
16     }
17
18     seg[node].data = NODE::merge(seg[seg[node].l].data, seg[seg[node].r].data);
19     return node;
20 }
```

```
1  int update(int prev_root, int L, int R, int idx, NODE v) { // Seg Persistente
2      int node = seg.size();
3      seg.push_back(prev_root == 0 ? Node() : seg[prev_root]);
4
5      if (L == R) { seg[node].data.apply(v); return node }
6
7      int mid = L + (R - L) / 2;
8
9      if (idx ≤ mid) { // Cuidado com ordem de avaliacao (LHS) pre-C++17
10         int prev_left = (prev_root == 0) ? 0 : seg[prev_root].l;
11         seg[node].l = update(prev_left, L, mid, idx, v);
12     }
13     else { // Cuidado com ordem de avaliacao (LHS) pre-C++17
14         int prev_right = (prev_root == 0) ? 0 : seg[prev_root].r;
15         seg[node].r = update(prev_right, mid + 1, R, idx, v);
16     }
17
18     seg[node].data = NODE::merge(seg[seg[node].l].data, seg[seg[node].r].data);
19     return node;
20 }
```



```
1  int update(int prev_root, int L, int R, int idx, NODE v) { // Seg Persistente
2      int node = seg.size();
3      seg.push_back(prev_root == 0 ? Node() : seg[prev_root]);
4
5      if (L == R) { seg[node].data.apply(v); return node }
6
7      int mid = L + (R - L) / 2;
8
9      if (idx ≤ mid) { // Cuidado com ordem de avaliacao (LHS) pre-C++17
10         int prev_left = (prev_root == 0) ? 0 : seg[prev_root].l;
11         seg[node].l = update(prev_left, L, mid, idx, v);
12     }
13     else { // Cuidado com ordem de avaliacao (LHS) pre-C++17
14         int prev_right = (prev_root == 0) ? 0 : seg[prev_root].r;
15         seg[node].r = update(prev_right, mid + 1, R, idx, v);
16     }
17
18     seg[node].data = NODE::merge(seg[seg[node].l].data, seg[seg[node].r].data);
19     return node;
20 }
```

```
1  int update(int prev_root, int L, int R, int idx, NODE v) { // Seg Persistente
2      int node = seg.size();
3      seg.push_back(prev_root == 0 ? Node() : seg[prev_root]);
4
5      if (L == R) { seg[node].data.apply(v); return node }
6
7      int mid = L + (R - L) / 2;
8
9      if (idx ≤ mid) { // Cuidado com ordem de avaliacao (LHS) pre-C++17
10         int prev_left = (prev_root == 0) ? 0 : seg[prev_root].l;
11         seg[node].l = update(prev_left, L, mid, idx, v);
12     }
13     else { // Cuidado com ordem de avaliacao (LHS) pre-C++17
14         int prev_right = (prev_root == 0) ? 0 : seg[prev_root].r;
15         seg[node].r = update(prev_right, mid + 1, R, idx, v);
16     }
17
18     seg[node].data = NODE::merge(seg[seg[node].l].data, seg[seg[node].r].data);
19     return node;
20 }
```

```
1  int update(int prev_root, int L, int R, int idx, NODE v) { // Seg Persistente
2      int node = seg.size();
3      seg.push_back(prev_root == 0 ? Node() : seg[prev_root]);
4
5      if (L == R) { seg[node].data.apply(v); return node }
6
7      int mid = L + (R - L) / 2;
8
9      if (idx ≤ mid) { // Cuidado com ordem de avaliacao (LHS) pre-C++17
10         int prev_left = (prev_root == 0) ? 0 : seg[prev_root].l;
11         seg[node].l = update(prev_left, L, mid, idx, v);
12     }
13     else { // Cuidado com ordem de avaliacao (LHS) pre-C++17
14         int prev_right = (prev_root == 0) ? 0 : seg[prev_root].r;
15         seg[node].r = update(prev_right, mid + 1, R, idx, v);
16     }
17
18     seg[node].data = NODE::merge(seg[seg[node].l].data, seg[seg[node].r].data);
19     return node;
20 }
```

```
1  int update(int prev_root, int L, int R, int idx, NODE v) { // Seg Persistente
2      int node = seg.size();
3      seg.push_back(prev_root == 0 ? Node() : seg[prev_root]);
4
5      if (L == R) { seg[node].data.apply(v); return node }
6
7      int mid = L + (R - L) / 2;
8
9      if (idx ≤ mid) { // Cuidado com ordem de avaliacao (LHS) pre-C++17
10         int prev_left = (prev_root == 0) ? 0 : seg[prev_root].l;
11         seg[node].l = update(prev_left, L, mid, idx, v);
12     }
13     else { // Cuidado com ordem de avaliacao (LHS) pre-C++17
14         int prev_right = (prev_root == 0) ? 0 : seg[prev_root].r;
15         seg[node].r = update(prev_right, mid + 1, R, idx, v);
16     }
17
18     seg[node].data = NODE::merge(seg[seg[node].l].data, seg[seg[node].r].data);
19     return node;
20 }
```

```
1  NODE query(int v, ll l, ll r, ll ql, ll qr) { // Seg Esparsa
2      if (v == -1 || ql > r || qr < l) return NODE();
3      if (ql ≤ l && r ≤ qr) return seg[v].node;
4      push(v, l, r);
5      ll mid = l + (r - l) / 2;
6      return NODE::merge(query(seg[v].l, l, mid, ql, qr),
7                          query(seg[v].r, mid + 1, r, ql, qr));
8  }
9
10 void update(ll ql, ll qr, const TAG& t) { update(0, L, R, ql, qr, t); }
11 NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
```

```
1  NODE query(int node, int L, int R, int i, int j) { // Seg Persistente
2      if (node == 0 || i > R || j < L) return NODE();
3      if (i ≤ L && R ≤ j) return st[node].data;
4      int mid = L + (R - L) / 2;
5      return NODE::merge(query(st[node].l, L, mid, i, j),
6                          query(st[node].r, mid + 1, R, i, j));
7  }
8
9  void update(int idx, NODE v) {
10     roots.push_back(update(roots.back(), 0, N - 1, idx, v));
11 }
12
13 NODE query(int version, int l, int r) {
14     return query(roots[version], 0, N - 1, l, r);
15 }
```

```
1  NODE query(int node, int L, int R, int i, int j) { // Seg Persistente
2      if (node == 0 || i > R || j < L) return NODE();
3      if (i ≤ L && R ≤ j) return st[node].data;
4      int mid = L + (R - L) / 2;
5      return NODE::merge(query(st[node].l, L, mid, i, j),
6                          query(st[node].r, mid + 1, R, i, j));
7  }
8
9  void update(int idx, NODE v) {
10     roots.push_back(update(roots.back(), 0, N - 1, idx, v));
11 }
12
13 NODE query(int version, int l, int r) {
14     return query(roots[version], 0, N - 1, l, r);
15 }
```

```
1  NODE query(int node, int L, int R, int i, int j) { // Seg Persistente
2      if (node == 0 || i > R || j < L) return NODE();
3      if (i ≤ L && R ≤ j) return st[node].data;
4      int mid = L + (R - L) / 2;
5      return NODE::merge(query(st[node].l, L, mid, i, j),
6                          query(st[node].r, mid + 1, R, i, j));
7  }
8
9  void update(int idx, NODE v) {
10     roots.push_back(update(roots.back(), 0, N - 1, idx, v));
11 }
12
13 NODE query(int version, int l, int r) {
14     return query(roots[version], 0, N - 1, l, r);
15 }
```



```
1  NODE query(int node, int L, int R, int i, int j) { // Seg Persistente
2      if (node == 0 || i > R || j < L) return NODE();
3      if (i ≤ L && R ≤ j) return st[node].data;
4      int mid = L + (R - L) / 2;
5      return NODE::merge(query(st[node].l, L, mid, i, j),
6                          query(st[node].r, mid + 1, R, i, j));
7  }
8
9  void update(int idx, NODE v) {
10     roots.push_back(update(roots.back(), 0, N - 1, idx, v));
11 }
12
13 NODE query(int version, int l, int r) {
14     return query(roots[version], 0, N - 1, l, r);
15 }
```

```
1  NODE query(int node, int L, int R, int i, int j) { // Seg Persistente
2      if (node == 0 || i > R || j < L) return NODE();
3      if (i ≤ L && R ≤ j) return st[node].data;
4      int mid = L + (R - L) / 2;
5      return NODE::merge(query(st[node].l, L, mid, i, j),
6                          query(st[node].r, mid + 1, R, i, j));
7  }
8
9  void update(int idx, NODE v) {
10     roots.push_back(update(roots.back(), 0, N - 1, idx, v));
11 }
12
13 NODE query(int version, int l, int r) {
14     return query(roots[version], 0, N - 1, l, r);
15 }
```

```
1  NODE query(int node, int L, int R, int i, int j) { // Seg Persistente
2      if (node == 0 || i > R || j < L) return NODE();
3      if (i ≤ L && R ≤ j) return st[node].data;
4      int mid = L + (R - L) / 2;
5      return NODE::merge(query(st[node].l, L, mid, i, j),
6                          query(st[node].r, mid + 1, R, i, j));
7  }
8
9  void update(int idx, NODE v) {
10     roots.push_back(update(roots.back(), 0, N - 1, idx, v));
11 }
12
13 NODE query(int version, int l, int r) {
14     return query(roots[version], 0, N - 1, l, r);
15 }
```

```
1  NODE query(int node, int L, int R, int i, int j) { // Seg Persistente
2      if (node == 0 || i > R || j < L) return NODE();
3      if (i ≤ L && R ≤ j) return st[node].data;
4      int mid = L + (R - L) / 2;
5      return NODE::merge(query(st[node].l, L, mid, i, j),
6                          query(st[node].r, mid + 1, R, i, j));
7  }
8
9  void update(int idx, NODE v) {
10     roots.push_back(update(roots.back(), 0, N - 1, idx, v));
11 }
12
13 NODE query(int version, int l, int r) {
14     return query(roots[version], 0, N - 1, l, r);
15 }
```

```
1  NODE query(int node, int L, int R, int i, int j) { // Seg Persistente
2      if (node == 0 || i > R || j < L) return NODE();
3      if (i ≤ L && R ≤ j) return st[node].data;
4      int mid = L + (R - L) / 2;
5      return NODE::merge(query(st[node].l, L, mid, i, j),
6                          query(st[node].r, mid + 1, R, i, j));
7  }
8
9  void update(int idx, NODE v) {
10     roots.push_back(update(roots.back(), 0, N - 1, idx, v));
11 }
12
13 NODE query(int version, int l, int r) {
14     return query(roots[version], 0, N - 1, l, r);
15 }
```

Lazy?

Nota: Lazy Propagation em SegTree Persistente é possível, mas exige criar novos nós também durante o push. Isso aumenta drasticamente o uso de memória e a complexidade. Evite a menos que seja estritamente necessário.



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Caso tenha interesse, assista a aula do primeiro dia da SummerCamp 2026 da turma World Finals!

The banner for the Brazilian ICPC Summer School 2026 features a festive design with colorful balloons and a sunburst logo. The text "Brazilian ICPC Summer School 2026" is prominently displayed in orange. Below the title, it states the dates "January 19th - 31st" and the location "Institute of Computing/UNICAMP - Campinas". A large red YouTube play button icon is centered. The banner also lists sponsors: "Promoted by" (SBC and Maratona de Programação), "Organized by" (Instituto de Computação and UNICAMP), "Premium Sponsor" (HUAWEI), "Gold Sponsors" (btgpactual, Instituto de Computação, and UNICAMP), "Silver Sponsors" (canals and neon), and "Bronze Sponsor" (Google).

Dúvidas?

Segment Tree Beats

Queries "impossíveis"

Até agora, lidamos com operações "amigáveis" que permitem Lazy Propagation:

- Range Add (+)
- Range Set (=)
- Range Mul (×)

Mas e se a operação não for distributiva ou for difícil de compor?

Imagine operações como:

- Range Chmin: $A_i = \min(A_i, x)$
- Range Modulo: $A_i = A_i \% M$
- Range Sqrt: $A_i = \sqrt{A_i}$

Por que o Lazy falha? Não existe uma fórmula fechada simples para calcular a soma de um intervalo após aplicar mod M ou sqrt sem olhar os elementos individualmente.

SegTree Beats

A ideia central da Segment Tree Beats é usar a Complexidade Amortizada.

Não usamos Lazy tradicional. Em vez disso, usamos condições de parada inteligentes (break_condition e tag_condition) para evitar propagar operações que não afetariam o resultado ou que já podem ser pre-calculadas.

```
1 void update(int no, int l, int r, int a, int b, const TAG& v) {
2     if (b < l || r < a) return;
3     if (a ≤ l && r ≤ b) {
4         seg[no].apply(v, l, r);
5         lazy[no].compose(v);
6         return;
7     }
8     push(no, l, r);
9     int m = (l + r) >> 1;
10    update(no << 1, l, m, a, b, v);
11    update((no << 1) | 1, m + 1, r, a, b, v);
12    seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
13 }
```

SegTree Beats

A ideia central da Segment Tree Beats é usar a Complexidade Amortizada.

Não usamos Lazy tradicional. Em vez disso, usamos condições de parada inteligentes (`break_condition` e `tag_condition`) para evitar propagar operações que não afetariam o resultado ou que já podem ser pre-calculadas.

```
1 void update(int no, int l, int r, int a, int b, const TAG& v) {
2     if (b < l || r < a || seg[no].break_condition(v)) return;
3     if (l == r) return seg[no].apply(v, l, r), void();
4     if ((a ≤ l && r ≤ b) && seg[no].tag_condition(v)) {
5         seg[no].apply(v, l, r);
6         lazy[no].compose(v);
7         return;
8     }
9     push(no, l, r);
10    int m = (l + r) >> 1;
11    update(no << 1, l, m, a, b, v);
12    update((no << 1) | 1, m + 1, r, a, b, v);
13    seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
14 }
```

SegTree Beats

A ideia central da Segment Tree Beats é usar a Complexidade Amortizada.

Não usamos Lazy tradicional. Em vez disso, usamos condições de parada inteligentes (`break_condition` e `tag_condition`) para evitar propagar operações que não afetariam o resultado ou que já podem ser pre-calculadas.

```
1 void update(int no, int l, int r, int a, int b, const TAG& v) {
2     if (b < l || r < a || seg[no].break_condition(v)) return;
3     if (l == r) return seg[no].apply(v, l, r), void();
4     if ((a ≤ l && r ≤ b) && seg[no].tag_condition(v)) {
5         seg[no].apply(v, l, r);
6         lazy[no].compose(v);
7         return;
8     }
9     push(no, l, r);
10    int m = (l + r) >> 1;
11    update(no << 1, l, m, a, b, v);
12    update((no << 1) | 1, m + 1, r, a, b, v);
13    seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
14 }
```

SegTree Beats

A ideia central da Segment Tree Beats é usar a Complexidade Amortizada.

Não usamos Lazy tradicional. Em vez disso, usamos condições de parada inteligentes (`break_condition` e `tag_condition`) para evitar propagar operações que não afetariam o resultado ou que já podem ser pre-calculadas.

```
1 void update(int no, int l, int r, int a, int b, const TAG& v) {
2     if (b < l || r < a || seg[no].break_condition(v)) return;
3     if (l == r) return seg[no].apply(v, l, r), void();
4     if ((a ≤ l && r ≤ b) && seg[no].tag_condition(v)) {
5         seg[no].apply(v, l, r);
6         lazy[no].compose(v);
7         return;
8     }
9     push(no, l, r);
10    int m = (l + r) >> 1;
11    update(no << 1, l, m, a, b, v);
12    update((no << 1) | 1, m + 1, r, a, b, v);
13    seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
14 }
```

SegTree Beats

A ideia central da Segment Tree Beats é usar a Complexidade Amortizada.

Não usamos Lazy tradicional. Em vez disso, usamos condições de parada inteligentes (`break_condition` e `tag_condition`) para evitar propagar operações que não afetariam o resultado ou que já podem ser pre-calculadas.

```
1 void update(int no, int l, int r, int a, int b, const TAG& v) {
2     if (b < l || r < a || seg[no].break_condition(v)) return;
3     if (l == r) return seg[no].apply(v, l, r), void();
4     if ((a ≤ l && r ≤ b) && seg[no].tag_condition(v)) {
5         seg[no].apply(v, l, r);
6         lazy[no].compose(v);
7         return;
8     }
9     push(no, l, r);
10    int m = (l + r) >> 1;
11    update(no << 1, l, m, a, b, v);
12    update((no << 1) | 1, m + 1, r, a, b, v);
13    seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
14 }
```

SegTree Beats

A Break Condition serve para evitar fazer updates desnecessários em segmentos que não serão afetados pela operação. Por exemplo, aplicar um $mod = x$, se o valor máximo do segmento já for menor que x .

Já a Tag Condition verifica se o segmento inteiro pode ser atualizado diretamente sem precisar propagar a operação para os filhos. Por exemplo, podemos ver $a \% x$ como $a - \lfloor \frac{a}{x} \rfloor \cdot x$. Sendo assim, se $\lfloor \frac{a}{x} \rfloor$ for constante no segmento, podemos aplicar a operação diretamente.

```
1 void update(int no, int l, int r, int a, int b, const TAG& v) {
2     if (b < l || r < a || seg[no].break_condition(v)) return;
3     if (l == r) return seg[no].apply(v, l, r), void();
4     if ((a ≤ l && r ≤ b) && seg[no].tag_condition(v)) {
5         seg[no].apply(v, l, r);
6         lazy[no].compose(v);
7         return;
8     }
9     push(no, l, r);
10    int m = (l + r) >> 1;
11    update(no << 1, l, m, a, b, v);
12    update((no << 1) | 1, m + 1, r, a, b, v);
13    seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
14 }
```

Prova de Complexidade

Queremos mostrar que, apesar de termos que "descer" em alguns casos, a complexidade é $\mathcal{O}((n+q) \log n \log C)$, onde C é o maior valor do array.

Para isso, vamos definir $\phi(v)$ o potencial de um nó e Φ o potencial da árvore:

$$\phi(v) = \sum_{i=l[v]}^{r[v]} \log(arr[i] + 1)$$

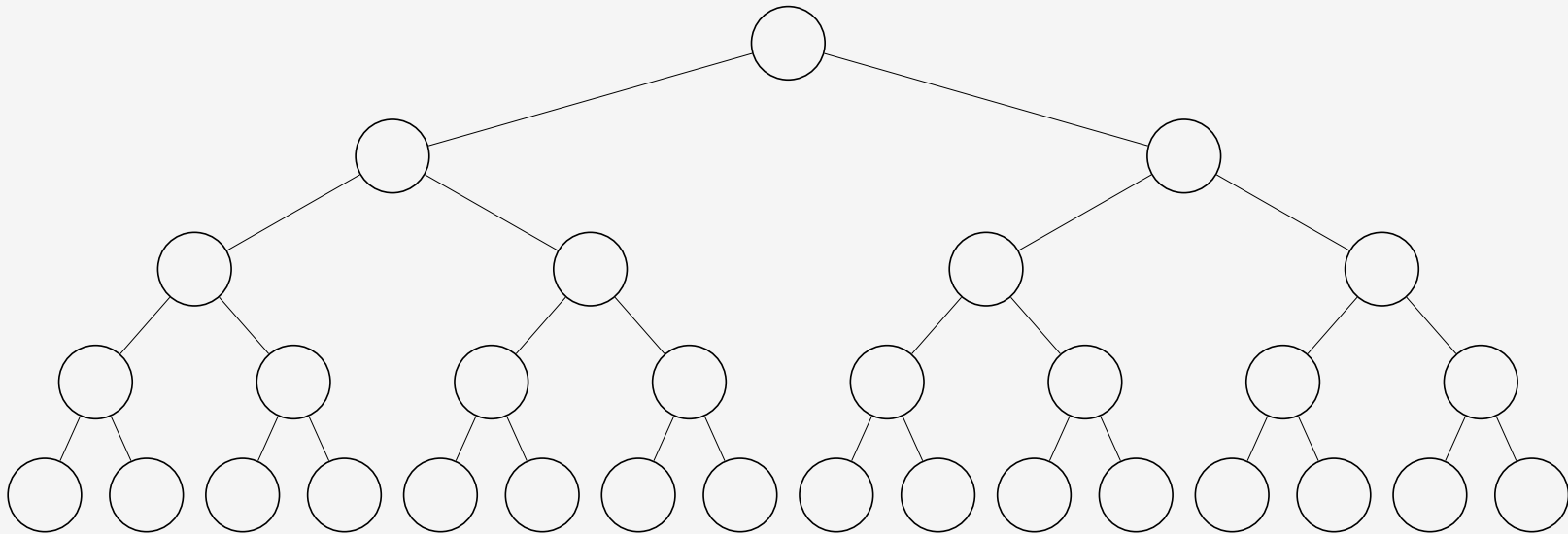
$$0 \leq \Phi = \sum \phi(v) \leq \mathcal{O}(n \log n \log C)$$

A desigualdade vale pois cada elemento contribui com no máximo $\log(C+1)$ para o potencial de um nó, cada um dos $\mathcal{O}(n)$ elementos está contido em exatamente um nó por nível e a árvore possui $\mathcal{O}(\log n)$ níveis.

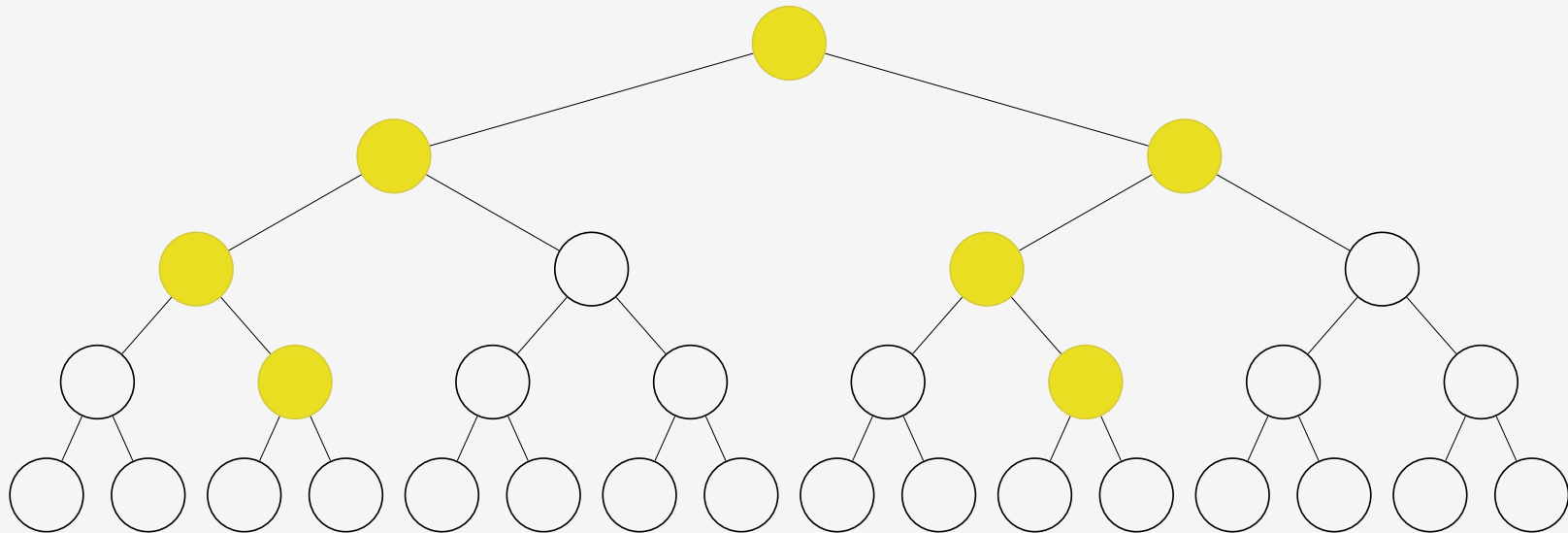
Além dessa definição de potencial, vamos separar os nós da árvore em três tipos:

- Simples: Nós que a árvore teria que descer de qualquer forma
- Terminais: Nós que param a descida (break condition ou tag condition)
- Não-terminais: Nós que a árvore desce por causa das condições especiais

Prova de Complexidade

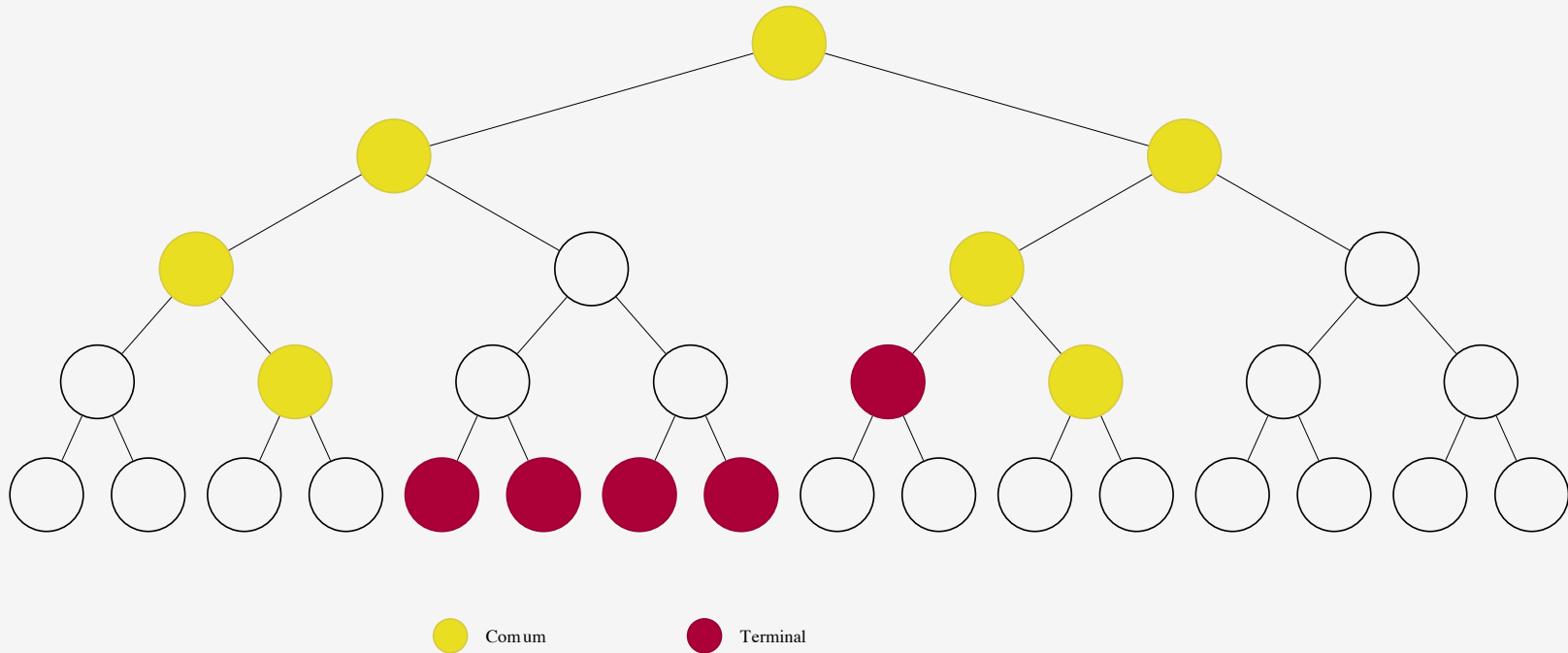


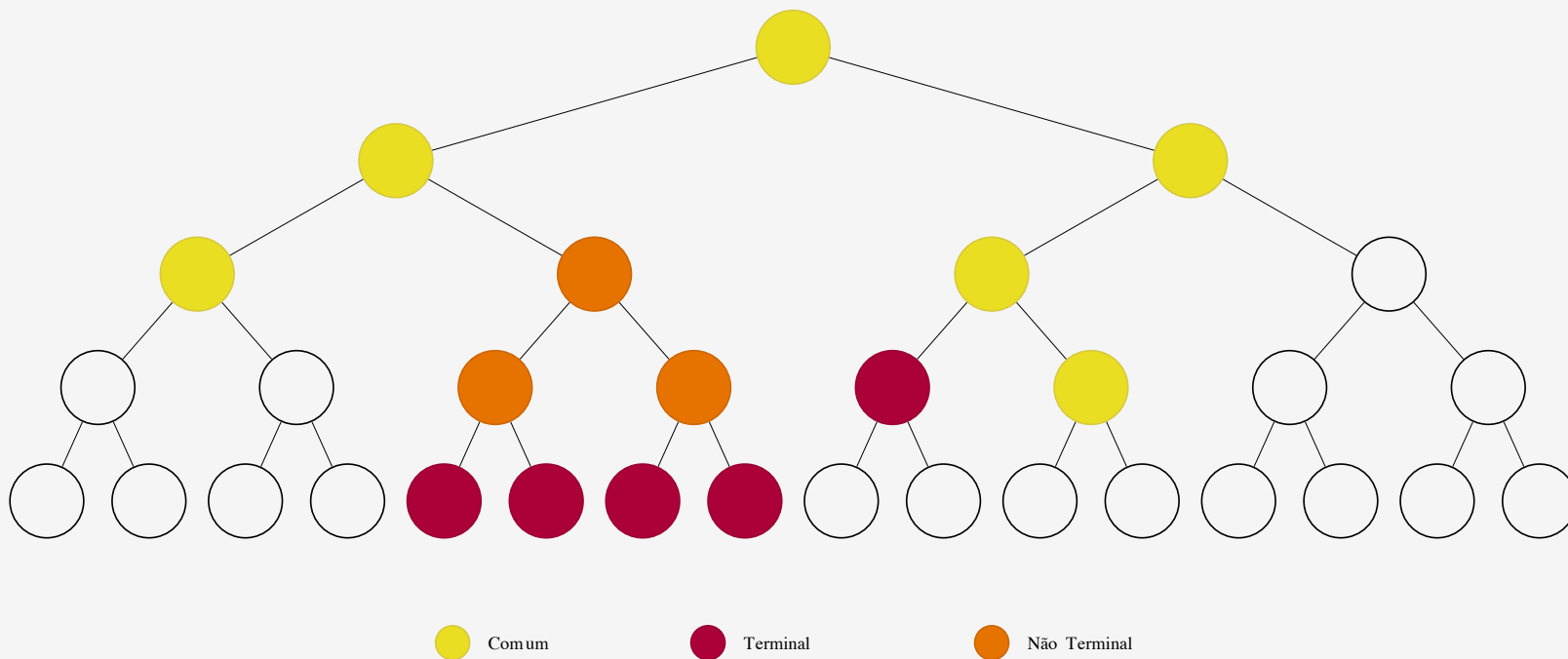
Prova de Complexidade



● Comum

Prova de Complexidade





Prova de Complexidade

Para efeitos de complexidade, toda vez que descemos para um nó terminal, terminamos uma descida, logo, não precisamos nos preocupar com esses nós.

Para os nós simples, a árvore teria que descer de qualquer forma, logo, eles também não impactam a complexidade extra. De qualquer forma, cada update só pode descer $\mathcal{O}(q \log n)$ vezes em nós simples.

Vamos mostrar que, ao visitar um nó não-terminal, o potencial da árvore diminui em pelo menos 1. Sendo assim, eles contribuem com $\mathcal{O}(n \log n \log c) + \Phi_+$ onde Φ_+ é o potencial adicional da árvore (vamos falar mais sobre posteriormente).

Definindo dessa forma, a complexidade total de descidas seria $\mathcal{O}(n \log n \log C + \Phi_+ + q \log n)$. Portanto, para provar que o algoritmo está em $\mathcal{O}((n + q) \log n \log C)$, precisamos mostrar que $\Phi_+ \in \mathcal{O}((n + q) \log n \log C)$.

Prova de Complexidade

Para calcular Φ_+ , podemos usar do fato de que, em cada atualização, sabemos que $oldValue \geq 0$ e que $newValue \leq C$. Logo, o potencial adicional de cada elemento é no máximo $\log(C + 1) - \log(0 + 1) = \log C$.

Sendo assim, como temos $\mathcal{O}(q)$ updates e cada update influencia em no máximo $\mathcal{O}(\log n)$ nós, temos $\Phi_+ \leq \mathcal{O}(q \log n \log C)$.

Agora, resta provar que o potencial diminui em pelo menos 1 para cada nó não-terminal que visitamos. Como o nó é não-terminal, algum filho dele tem que ser atualizado, logo, algum filho dele possui valor $k \geq x$.

Vamos usar a identidade $k \% x \leq \frac{k-1}{2}$ para mostrar que, como o valor de algum nó diminuiu pelo menos para a sua metade, o logaritmo do valor do nó diminui em pelo menos 1, logo, o potencial da árvore diminui em pelo menos 1.

SegTree Beats

Note que não usamos a `tag_condition` em lugar nenhum na prova. De fato, para esse caso com point-updates, a `tag_condition` não é necessária. No entanto, para casos mais gerais, como range-updates, a `tag_condition` é essencial para garantir a eficiência do algoritmo.

(O Peltorator não conseguiu explicar a prova mas ele jura que precisa)

```
1  struct Tag {
2      ll set, mod;
3
4      Tag(ll set = NO_OP, ll mod = NO_OP) : set(set), mod(mod) {}
5
6      void inline compose(const Tag& t) {
7          if (set != NO_OP) {
8              if (t.set != NO_OP) set = t.set, mod = NO_OP;
9              else set %= t.mod;
10         }
11         else {
12             if (t.set != NO_OP) set = t.set, mod = NO_OP;
13             else if (mod == NO_OP) mod = t.mod;
14             // CASO BEATS, não dá pra compor
15             // if (mod != NO_OP && t.mod != NO_OP)
16         }
17     }
18 };
```

SegTree Beats

Note que não usamos a `tag_condition` em lugar nenhum na prova. De fato, para esse caso com point-updates, a `tag_condition` não é necessária. No entanto, para casos mais gerais, como range-updates, a `tag_condition` é essencial para garantir a eficiência do algoritmo.

(O Peltorator não conseguiu explicar a prova mas ele jura que precisa)

```
1  struct Tag {
2      ll set, mod;
3
4      Tag(ll set = NO_OP, ll mod = NO_OP) : set(set), mod(mod) {}
5
6      void inline compose(const Tag& t) {
7          if (set != NO_OP) {
8              if (t.set != NO_OP) set = t.set, mod = NO_OP;
9              else set %= t.mod;
10         }
11         else {
12             if (t.set != NO_OP) set = t.set, mod = NO_OP;
13             else if (mod == NO_OP) mod = t.mod;
14             // CASO BEATS, não dá pra compor
15             // if (mod != NO_OP && t.mod != NO_OP)
16         }
17     }
18 };
```


SegTree Beats

Note que não usamos a `tag_condition` em lugar nenhum na prova. De fato, para esse caso com point-updates, a `tag_condition` não é necessária. No entanto, para casos mais gerais, como range-updates, a `tag_condition` é essencial para garantir a eficiência do algoritmo.

(O Peltorator não conseguiu explicar a prova mas ele jura que precisa)

```
1  struct Tag {
2      ll set, mod;
3
4      Tag(ll set = NO_OP, ll mod = NO_OP) : set(set), mod(mod) {}
5
6      void inline compose(const Tag& t) {
7          if (set != NO_OP) {
8              if (t.set != NO_OP) set = t.set, mod = NO_OP;
9              else set %= t.mod;
10         }
11         else {
12             if (t.set != NO_OP) set = t.set, mod = NO_OP;
13             else if (mod == NO_OP) mod = t.mod;
14             // CASO BEATS, não dá pra compor
15             // if (mod != NO_OP && t.mod != NO_OP)
16         }
17     }
18 };
```

SegTree Beats

Note que não usamos a `tag_condition` em lugar nenhum na prova. De fato, para esse caso com point-updates, a `tag_condition` não é necessária. No entanto, para casos mais gerais, como range-updates, a `tag_condition` é essencial para garantir a eficiência do algoritmo.

(O Peltorator não conseguiu explicar a prova mas ele jura que precisa)

```
1  struct Tag {
2      ll set, mod;
3
4      Tag(ll set = NO_OP, ll mod = NO_OP) : set(set), mod(mod) {}
5
6      void inline compose(const Tag& t) {
7          if (set != NO_OP) {
8              if (t.set != NO_OP) set = t.set, mod = NO_OP;
9              else set %= t.mod;
10         }
11         else {
12             if (t.set != NO_OP) set = t.set, mod = NO_OP;
13             else if (mod == NO_OP) mod = t.mod;
14             // CASO BEATS, não dá pra compor
15             // if (mod != NO_OP && t.mod != NO_OP)
16         }
17     }
18 };
```

SegTree Beats

Note que não usamos a `tag_condition` em lugar nenhum na prova. De fato, para esse caso com point-updates, a `tag_condition` não é necessária. No entanto, para casos mais gerais, como range-updates, a `tag_condition` é essencial para garantir a eficiência do algoritmo.

(O Peltorator não conseguiu explicar a prova mas ele jura que precisa)

```
1  struct Tag {
2      ll set, mod;
3
4      Tag(ll set = NO_OP, ll mod = NO_OP) : set(set), mod(mod) {}
5
6      void inline compose(const Tag& t) {
7          if (set != NO_OP) {
8              if (t.set != NO_OP) set = t.set, mod = NO_OP;
9              else set %= t.mod;
10         }
11         else {
12             if (t.set != NO_OP) set = t.set, mod = NO_OP;
13             else if (mod == NO_OP) mod = t.mod;
14             // CASO BEATS, não dá pra compor
15             // if (mod != NO_OP && t.mod != NO_OP)
16         }
17     }
18 };
```

SegTree Beats

Note que não usamos a `tag_condition` em lugar nenhum na prova. De fato, para esse caso com point-updates, a `tag_condition` não é necessária. No entanto, para casos mais gerais, como range-updates, a `tag_condition` é essencial para garantir a eficiência do algoritmo.

(O Peltorator não conseguiu explicar a prova mas ele jura que precisa)

```
1  struct Tag {
2      ll set, mod;
3
4      Tag(ll set = NO_OP, ll mod = NO_OP) : set(set), mod(mod) {}
5
6      void inline compose(const Tag& t) {
7          if (set != NO_OP) {
8              if (t.set != NO_OP) set = t.set, mod = NO_OP;
9              else set %= t.mod;
10         }
11         else {
12             if (t.set != NO_OP) set = t.set, mod = NO_OP;
13             else if (mod == NO_OP) mod = t.mod;
14             // CASO BEATS, não dá pra compor
15             // if (mod != NO_OP && t.mod != NO_OP)
16         }
17     }
18 };
```

```
1  struct Node {
2      ll val, mx;
3
4      Node(ll val = 0) : val(val), mx(val) {}
5
6      inline static Node merge(const Node& l, const Node& r) {
7          return {l.val + r.val, max(l.mx, r.mx)};
8      }
9
10     inline void apply(const Tag& t, ll l, ll r) {
11         if (t.set != NO_OP) {
12             mx = t.set;
13             val = t.set * (r-l+1);
14         }
15         else if (t.mod != NO_OP) {
16             mx %= t.mod;
17             val = mx * (r-l+1);
18         }
19     }
```

```
1  struct Node {
2      ll val, mx;
3
4      Node(ll val = 0) : val(val), mx(val) {}
5
6      inline static Node merge(const Node& l, const Node& r) {
7          return {l.val + r.val, max(l.mx, r.mx)};
8      }
9
10     inline void apply(const Tag& t, ll l, ll r) {
11         if (t.set != NO_OP) {
12             mx = t.set;
13             val = t.set * (r-l+1);
14         }
15         else if (t.mod != NO_OP) {
16             mx %= t.mod;
17             val = mx * (r-l+1);
18         }
```

```
1 struct Node {
2     ll val, mx;
3
4     Node(ll val = 0) : val(val), mx(val) {}
5
6     inline static Node merge(const Node& l, const Node& r) {
7         return {l.val + r.val, max(l.mx, r.mx)};
8     }
9
10    inline void apply(const Tag& t, ll l, ll r) {
11        if (t.set != NO_OP) {
12            mx = t.set;
13            val = t.set * (r-l+1);
14        }
15        else if (t.mod != NO_OP) {
16            mx %= t.mod;
17            val = mx * (r-l+1);
18        }
```

```
6     inline static Node merge(const Node& l, const Node& r) {
7         return {l.val + r.val, max(l.mx, r.mx)};
8     }
9
10    inline void apply(const Tag& t, ll l, ll r) {
11        if (t.set != NO_OP) {
12            mx = t.set;
13            val = t.set * (r-l+1);
14        }
15        else if (t.mod != NO_OP) {
16            mx %= t.mod;
17            val = mx * (r-l+1);
18        }
19    }
20
21    inline bool break_condition(const Tag& t) const {
22        return t.set == NO_OP && mx < t.mod;
23    }
24
25    inline void push(const Tag& t, ll l, ll r) {
```



```
4     Node(ll val = 0) : val(val), mx(val) {}
5
6     inline static Node merge(const Node& l, const Node& r) {
7         return {l.val + r.val, max(l.mx, r.mx)};
8     }
9
10    inline void apply(const Tag& t, ll l, ll r) {
11        if (t.set != NO_OP) {
12            mx = t.set;
13            val = t.set * (r-l+1);
14        }
15        else if (t.mod != NO_OP) {
16            mx %= t.mod;
17            val = mx * (r-l+1);
18        }
19    }
20
21    inline bool break_condition(const Tag& t) const {
22        return t.set == NO_OP && mx < t.mod;
```

```
8     }
9
10    inline void apply(const Tag& t, ll l, ll r) {
11        if (t.set != NO_OP) {
12            mx = t.set;
13            val = t.set * (r-l+1);
14        }
15        else if (t.mod != NO_OP) {
16            mx %= t.mod;
17            val = mx * (r-l+1);
18        }
19    }
20
21    inline bool break_condition(const Tag& t) const {
22        return t.set == NO_OP && mx < t.mod;
23    }
24
25    inline bool tag_condition(const Tag& t) const {
26        return false;
27    }
```

```
11     if (t.set  $\neq$  NO_OP) {
12         mx = t.set;
13         val = t.set * (r-l+1);
14     }
15     else if (t.mod  $\neq$  NO_OP) {
16         mx %= t.mod;
17         val = mx * (r-l+1);
18     }
19 }
20
21 inline bool break_condition(const Tag& t) const {
22     return t.set == NO_OP && mx < t.mod;
23 }
24
25 inline bool tag_condition(const Tag& t) const {
26     return false;
27 }
28 };
```

```
11     if (t.set  $\neq$  NO_OP) {
12         mx = t.set;
13         val = t.set * (r-l+1);
14     }
15     else if (t.mod  $\neq$  NO_OP) {
16         mx %= t.mod;
17         val = mx * (r-l+1);
18     }
19 }
20
21 inline bool break_condition(const Tag& t) const {
22     return t.set == NO_OP && mx < t.mod;
23 }
24
25 inline bool tag_condition(const Tag& t) const {
26     return false;
27 }
28 };
```

```
1  struct Node {
2      ll val, mx;
3
4      Node(ll val = 0) : val(val), mx(val) {}
5
6      inline static Node merge(const Node& l, const Node& r) {
7          return {l.val + r.val, max(l.mx, r.mx)};
8      }
9
10     inline void apply(const Tag& t, ll l, ll r) {
11         if (t.set != NO_OP) {
12             mx = t.set;
13             val = t.set * (r-l+1);
14         }
15         else if (t.mod != NO_OP) {
16             mx %= t.mod;
17             val = mx * (r-l+1);
18         }
19     }
```

Outras operações

No geral, a prova de complexidade para qualquer operação de Seg Beats é similar. Definimos um potencial inicial Φ e um potencial adicional Φ_+ , e mostramos que o potencial diminui em pelo menos 1 para cada nó não-terminal visitado e que Φ_+ é limitado por uma função polinomial de n e q .

É como se tivéssemos um "orçamento" de descidas e cada operação que fazemos gasta um pouco dele. A chave é mostrar que, mesmo com descidas extras, o orçamento total necessário ainda é limitado por uma função polinomial de n e q .

Em especial, existem mais 3 operações "famosas" de seg beats:

- $\text{chmin}(A_i = \min(A_i, x))$ ("Ji Driver Segment Tree")
- chmin , chmax and add ("Extended Ji Driver Segment Tree")
- chmin and/or add update with gcd query (não veremos, mas não é difícil de chegar)

CHMIN "Ji Driver Segment Tree"

Queries que queremos suportar:

- Update Range Chmin: $A_i = \min(A_i, x)$
- Range Sum: $S = \sum_{i=L}^R A_i$

Sendo assim, podemos definir:

- Break Condition: $\max[v] \leq x$
- Tag Condition: $\text{secondMax}[v] < x < \max[v]$

Com essa tag condition, se soubermos a quantidade de elementos que são iguais ao máximo, podemos calcular o valor total do intervalo após a operação de chmin sem precisar olhar os elementos individualmente.

$$\text{newSum} = \text{Sum} - (\max[v] - x) \cdot \text{countMax}[v]$$

A composição de Tags também funciona, visto que se quisermos fazer um chmin com x_1 e depois um chmin com x_2 , o resultado é o mesmo que fazer um chmin com $\min(x_1, x_2)$.

Prova de Complexidade

Vamos definir $\phi(v)$ o potencial de um nó e Φ o potencial da árvore:

$$\phi(v) = \text{número de elementos distintos em } [L_v, R_v]$$

$$0 \leq \Phi = \sum \phi(v) \leq \mathcal{O}(n \log n)$$

A desigualdade vale pois cada um dos n elementos é contabilizado no potencial de, no máximo, $\mathcal{O}(\log n)$ nós (os seus ancestrais).

Análise do Potencial ($\Delta\Phi$):

- Terminais: Mantém ($\Delta\Phi = 0$)
 - Mapeamento 1:1 (Ex: todos $x \rightarrow y$).

- Não-Terminais: Diminui ($\Delta\Phi < 0$)
 - Fusão de valores distintos reduz a contagem.
- Simples: Varia ($\Delta\Phi = ?$)
 - Pode quebrar grupos iguais (aumenta) ou unir valores distintos (diminui).

Note que, no caso dos nós simples, o aumento total é de $\mathcal{O}(\log n)$ por update ou $\mathcal{O}(q \log n)$ no total. Logo, $\Phi_+ \in \mathcal{O}(q \log n)$

Prova de Complexidade

Agora basta provar que, visitando um nó não-terminal o potencial é diminuído em pelo menos um. Isso acontece por que, se o nó é um nó não terminal, então $\max < \text{secondMax} < x$, logo, ambos $\max$ e secondMax vão colapsar para o mesmo valor x e diminuir o potencial em pelo menos 1.

Portanto, a complexidade total do algoritmo é:

- $\mathcal{O}(n \log n)$: Do potencial inicial
- $\mathcal{O}(q \log n)$: De visitar os nós comuns
- **Total:** $\mathcal{O}((n + q) \log n)$



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



MaratonaCIn

Note que, de forma análoga, podemos suportar a operação de $\text{chmax}(A_i = \max(A_i, x))$.

Inclusive, é possível suportar ambas as operações simultaneamente guardando o máximo, 2º máximo, o mínimo, 2º mínimo, a quantidade de máximos e a quantidade de mínimos para cada intervalo.

Uma operação de $\text{assign}(=)$ também pode ser adicionada sem alterar a complexidade temporal pois também soma, no máximo, $\mathcal{O}(q \log n)$ ao Φ_+ .

Extended Ji Driver Segment Tree

Queries que queremos suportar:

- Update Range Chmin: $A_i = \min(A_i, x)$
- Update Range Chmax: $A_i = \max(A_i, x)$
- Update Range Assign: $A_i = x$
- Update Range Sum: $A_i += x$
- Range Sum: $S = \sum_{i=L}^R A_i$
- Range Min: $S = \min_{i=L}^R A_i$
- Range Max: $S = \max_{i=L}^R A_i$

Já sabemos como fazer todas essas operações simultaneamente, exceto pelo update de range com soma. Na prática, a operação de $(+)$ não é difícil de implementar, no entanto, ela quebra a nossa complexidade do potencial.

Suponha um array A com n elementos. Suponha que os primeiros $\frac{n}{2}$ elementos sejam distintos e os próximos $\frac{n}{2}$ elementos sejam uma cópia dos primeiros. Agora suponha que fazemos a query $addone(\frac{n}{2}, n)$. Agora o array tem n elementos, portanto, $\Phi_+ \in \mathcal{O}(n)$.

Prova de Complexidade

Vamos redefinir o potencial de um nó $\phi(v)$ da seguinte forma:

$$\phi(v) = \begin{cases} 1 & \text{se } \max(\text{left}) \neq \max(\text{right}) \\ 0 & \text{caso contrário} \end{cases}$$

$$0 \leq \Phi(v) = \sum \phi(v) \leq \mathcal{O}(n)$$

A desigualdade vale pois existem no máximo $4 \cdot n$ nós que contribuem com no máximo 1 para o potencial.

Vamos chamar os nós não-terminais $u/\phi(u) = 1$ de marcados. Para cada nó marcado u , existe, na sua subárvore um nó v tal que $\text{left}(v) = \max(u), \text{right}(v) = \text{secondMax}(u)$.

Sendo assim, ao aplicar um $\text{chmin}(x)$, $\max(u) < \text{secondMax}(u) < x$, caso contrário, u não seria não-terminal.

Sendo assim, ao aplicar a operação, os dois filhos de v vão se tornar iguais a x , sendo assim, $\phi(v)$ muda de 1 para 0.

Com isso, acabamos de provar que o potencial continua diminuindo em pelo menos 1 toda vez que visitamos uma subtree que contem um nó não terminal. Como essa subtree contém $\mathcal{O}(\log n)$ nós, o potencial diminui em $\frac{1}{\log n}$ por nó não-terminal visitado.

Prova de Complexidade

Pela nossa definição, ao passar por nós comuns, o potencial pode ser modificado em no máximo $\mathcal{O}(q \log n)$ nós. Como o potencial de cada nó em no máximo 1, $\Phi_+ \in \mathcal{O}(q \log n)$

Como ao passar por um nó não-terminal, o potencial diminui em $\frac{1}{\log n}$, vamos multiplicar todo o potencial por $\log n$ e obter, agora um novo potencial Ψ que é diminuído em 1 ao passar por cada nó não-terminal.

$$\Psi \in \mathcal{O}(n \log n)$$

$$\Psi_+ \in \mathcal{O}(q \log^2 n)$$

Portanto, a complexidade total do algoritmo está em $\mathcal{O}((n + q) \log^2 n)$

Ou seja, a adição da operação $(+ =)$ altera a classe de complexidade da SegBeats por apenas um logaritmo, mantendo a eficiência do algoritmo.

Variante	Operações	Complexidade
Lazy SegTree	$+, =, \text{Sum}$	$\mathcal{O}(q \log n)$
Jiri Driver	$\min =, \text{Sum}$	$\mathcal{O}((n + q) \log n)$
Extended Jiri	$\min =, +, \text{Sum}$	$\mathcal{O}((n + q) \log^2 N)$

Dúvidas?

Dúvidas?

Próximo passo: Homework!

Créditos

- Todos os Códigos dessa aula foram tirados da minha biblioteca pessoal de C++
- Todos os Diagramas usados nessa apresentação foram feitos por mim usando Latex/TikZ
- A apresentação foi construída usando Slidev
- Diagramas de Segtrees Esparsas e Persistentes foram inspirados no trabalho do Justice Hui
- Provas e exemplos de Seg Beats são inspirados no trabalho do Peltorator



That's all, Folks!

